

PROJET DE FIN D'ÉTUDES

Pour l'obtention du titre de
Chef de Projet Web -- RNCP40857

Titre:

**SG SecureBank: Application pour la détection de fraude dans les transactions
des cartes bancaires en utilisant l'intelligence artificielle**

Entreprise:

Société Générale (SG)

Apprenante :

Yasmine FRIAA

Soutenu le 01/09/2026

Table des matières

1	L'analyse des besoins du client dans le cadre d'un projet digital	6
1.1	Contexte et objectifs stratégiques	7
1.1.1	Présentation de l'entreprise	7
1.1.2	Problématique	8
1.1.3	Opportunité de marché	8
1.1.4	Besoin identifié	8
1.1.5	Objectifs stratégiques du projet	8
1.1.6	Analyse SWOT	8
1.2	Analyse des besoins	9
1.2.1	Méthodologie de recueil des besoins	9
1.2.2	Analyse du processus métier	9
1.2.3	Résultats de l'analyse des besoins	10
1.2.4	Recensement des besoins	10
1.2.4.1	Besoins fonctionnels	10
1.2.4.2	Besoins techniques	11
1.2.4.3	Besoins liés aux données	11
1.2.5	Synthèse des besoins	11
1.2.6	Priorisation des besoins (méthode MoSCoW)	11
1.2.7	Veille technologique	11
1.2.7.1	Choix du framework back-end	12
1.2.7.2	Choix de la technologie front-end	12
1.2.7.3	Choix du système de gestion de base de données	12
1.2.8	Analyse des risques	12
1.2.9	Impacts environnementaux et sociaux	13
1.2.10	Évaluation globale de la faisabilité	14
1.3	Présentation du cahier des charges	14
1.3.1	Périmètre du projet	15
1.3.1.1	Fonctionnalités incluses	15
1.3.1.2	Fonctionnalités exclues	15
1.3.1.3	Objectifs fonctionnels	15
1.3.1.4	Contraintes du périmètre	16
1.3.2	Description fonctionnelle de la solution	16
1.3.2.1	Module d'authentification	16
1.3.2.2	Module de gestion des clients, comptes et transactions	16
1.3.2.3	Module de détection intelligente de fraude	16
1.3.2.4	Module de gestion des alertes	17
1.3.2.5	Tableau de bord de supervision	17
1.4	Architecture technique et choix technologiques	17
1.4.1	Architecture générale de la solution	17
1.4.2	Technologies utilisées	18

1.4.2.1	Technologies Front-end	18
1.4.2.2	Technologies Back-end	18
1.4.2.3	Base de données	18
1.4.2.4	Technologies Machine Learning	19
1.4.3	Sécurité de la solution	19
1.5	Solution d'hébergement et nom de domaine préconisé	19
1.5.1	Choix de la solution d'hébergement	19
1.5.2	Architecture d'hébergement proposée	20
1.5.3	Nom de domaine proposé	20
1.5.4	Justification du choix	20
1.6	Stratégie de sauvegarde	20
1.6.1	Types de sauvegarde	21
1.6.2	Stockage et sécurité des sauvegardes	21
1.6.3	Plan de restauration	21
1.7	Parties prenantes et gestion de projet	21
1.7.1	Identification des parties prenantes	21
1.7.2	Méthode de gestion de projet	22
1.8	Planification du projet, ressources humaines et budget	22
1.8.1	Rétroplanning du projet	22
1.8.2	Ressources humaines nécessaires	22
1.8.3	Estimation du budget	23
1.9	Identification des goulots d'étranglement et des risques de retard	24
1.9.1	Stratégie d'anticipation	24
2	La conception et le développement d'une solution web	26
2.1	Architecture technique	27
2.1.1	Brique Data	28
2.1.2	Collecte et gouvernance des données	29
2.1.3	Justification des choix techniques	30
2.2	Maquettes et prototypes	31
2.2.1	Interfaces principales de l'application	31
2.2.1.1	Page de connexion	31
2.2.1.2	Tableau de bord	32
2.2.1.3	Page transactions	32
2.2.1.4	Choix graphiques et ergonomiques	33
2.2.2	Respect des critères ergonomiques	33
2.2.2.1	Auto-évaluation et ajustements	33
2.2.3	Présentation du développement front-end	34
2.2.3.1	Choix techniques et justification	34
2.2.3.2	Approche mobile-first	35
2.2.3.3	Tests réalisés sur plusieurs résolutions	35
2.2.3.4	Limites identifiées	35
2.2.4	La présentation du développement back-end	36
2.2.4.1	Standards de programmation et bonnes pratiques	36
2.2.4.2	Schéma relationnel	37
2.2.4.3	Optimisations des requêtes SQL	37
2.2.4.4	Endpoints sécurisés	38
2.2.4.5	Documentation des API	38
2.2.4.6	Mesures de performance	38
2.2.4.7	Limites identifiées	39

2.2.5	Tests et qualité logicielle	39
2.2.5.1	Stratégie de tests	39
2.2.5.2	Bug détecté et corrigé grâce aux tests	40
2.2.5.3	Correction d'un second bug	41
2.2.5.4	Limites identifiées	41
2.2.5.5	Limites des tests	41
2.2.6	Conformité au RGPD	42
2.2.7	Accessibilité	42
2.2.8	Livrables	42
2.2.9	Processus de gestion des mises à jour et des incidents post-déploiement	43
2.2.9.1	Plan de mise à jour	43
2.2.9.2	Processus de déploiement	43
2.2.9.3	Gestion des incidents et scénarios d'alertes	44
2.2.10	Retours des parties prenantes et ajustements	44
2.2.10.1	Retours sur l'organisation et la planification	45
2.2.10.2	Retours sur l'interface et l'ergonomie	45
2.2.10.3	Retours sur la sécurité et la conformité	45
2.2.10.4	Retours sur la conformité RGPD	45
2.2.10.5	Synthèse	45
A	Annexes	47
A.1	Personas	47
A.1.1	Persona 1 : Analyste fraude	47
A.1.2	Persona 2 : Responsable conformité et sécurité	47
A.2	Compte-rendu d'atelier simulé	47
A.3	Cas d'utilisation principaux	48

Introduction Générale

La fraude financière représente aujourd'hui un défi majeur et croissant pour les institutions bancaires et les acteurs du secteur financier. Avec lessor des technologies numériques, la multiplication des moyens de paiement électroniques et la globalisation des échanges, les fraudeurs disposent de techniques toujours plus sophistiquées et évolutives, rendant la détection des comportements frauduleux plus complexe que jamais. Face à cette menace, il est devenu indispensable de concevoir des systèmes intelligents, capables d'analyser en temps réel des volumes massifs de données transactionnelles, tout en garantissant un haut niveau de précision dans l'identification des fraudes.

Face à la montée des menaces, les banques doivent adopter des solutions innovantes pour détecter rapidement les comportements suspects, limiter les pertes financières et sécuriser les données clients. Les technologies web, combinées à l'intelligence artificielle et au machine learning, permettent aujourd'hui d'automatiser la surveillance des transactions et d'aider les équipes de contrôle dans leurs décisions.

Dans ce contexte, l'objectif de ce projet est de créer et de mettre en place une application web sophistiquée pour la détection des fraudes bancaires, destinée à SG SecureBank. Ce cas d'utilisation a été élaboré en réponse aux défis concrets auxquels sont confrontées les grandes institutions bancaires, particulièrement en ce qui concerne la détection de la fraude, le respect de la réglementation et la gestion des risques. Cette application est conçue pour offrir aux analystes un outil interactif qui facilite la traçabilité des transactions, l'identification des opérations à risque élevé via un système de notation, la gestion des alertes de fraude et l'élaboration de rapports d'analyse. L'option suggérée inclut aussi des critères cruciaux liés à la sécurité, à la confidentialité et à la protection des données, en conformité avec le Règlement Général sur la Protection des Données (RGPD).

Ce document constitue le cahier des charges du projet. Il présente l'analyse des besoins du client, les défis rencontrés, les finalités visées, les exigences tant fonctionnelles que techniques, les technologies sélectionnées, ainsi que les restrictions liées à l'exécution. Il détaille aussi la conception et la réalisation de la solution en ligne suggérée, en soulignant les diverses phases qui ont mené à sa mise en application.

Ce projet vise à offrir une solution contemporaine, fiable et efficace qui satisfait les exigences du client, tout en mettant en évidence les aptitudes développées dans les secteurs de l'évaluation des besoins, de la conduite de projet, de la création d'applications web et de la réalisation de solutions digitales avant-gardistes.

L'analyse des besoins du client dans le cadre d'un projet digital

1.1	Contexte et objectifs stratégiques	7
1.1.1	Présentation de l'entreprise	7
1.1.2	Problématique	8
1.1.3	Opportunité de marché	8
1.1.4	Besoin identifié	8
1.1.5	Objectifs stratégiques du projet	8
1.1.6	Analyse SWOT	8
1.2	Analyse des besoins	9
1.2.1	Méthodologie de recueil des besoins	9
1.2.2	Analyse du processus métier	9
1.2.3	Résultats de l'analyse des besoins	10
1.2.4	Recensement des besoins	10
1.2.4.1	Besoins fonctionnels	10
1.2.4.2	Besoins techniques	11
1.2.4.3	Besoins liés aux données	11
1.2.5	Synthèse des besoins	11
1.2.6	Priorisation des besoins (méthode MoSCoW)	11
1.2.7	Veille technologique	11
1.2.7.1	Choix du framework back-end	12
1.2.7.2	Choix de la technologie front-end	12
1.2.7.3	Choix du système de gestion de base de données	12
1.2.8	Analyse des risques	12
1.2.9	Impacts environnementaux et sociaux	13
1.2.10	Évaluation globale de la faisabilité	14
1.3	Présentation du cahier des charges	14
1.3.1	Périmètre du projet	15
1.3.1.1	Fonctionnalités incluses	15
1.3.1.2	Fonctionnalités exclues	15
1.3.1.3	Objectifs fonctionnels	15
1.3.1.4	Contraintes du périmètre	16
1.3.2	Description fonctionnelle de la solution	16
1.3.2.1	Module d'authentification	16
1.3.2.2	Module de gestion des clients, comptes et transactions	16
1.3.2.3	Module de détection intelligente de fraude	16
1.3.2.4	Module de gestion des alertes	17
1.3.2.5	Tableau de bord de supervision	17

1.4	Architecture technique et choix technologiques	17
1.4.1	Architecture générale de la solution	17
1.4.2	Technologies utilisées	18
1.4.2.1	Technologies Front-end	18
1.4.2.2	Technologies Back-end	18
1.4.2.3	Base de données	18
1.4.2.4	Technologies Machine Learning	19
1.4.3	Sécurité de la solution	19
1.5	Solution d'hébergement et nom de domaine préconisé	19
1.5.1	Choix de la solution d'hébergement	19
1.5.2	Architecture d'hébergement proposée	20
1.5.3	Nom de domaine proposé	20
1.5.4	Justification du choix	20
1.6	Stratégie de sauvegarde	20
1.6.1	Types de sauvegarde	21
1.6.2	Stockage et sécurité des sauvegardes	21
1.6.3	Plan de restauration	21
1.7	Parties prenantes et gestion de projet	21
1.7.1	Identification des parties prenantes	21
1.7.2	Méthode de gestion de projet	22
1.8	Planification du projet, ressources humaines et budget	22
1.8.1	Rétroplanning du projet	22
1.8.2	Ressources humaines nécessaires	22
1.8.3	Estimation du budget	23
1.9	Identification des goulots d'étranglement et des risques de retard	24
1.9.1	Stratégie d'anticipation	24

1.1 Contexte et objectifs stratégiques

1.1.1 Présentation de l'entreprise

La Société Générale (SG), l'une des grandes banques françaises, opère à l'échelle mondiale dans la banque de détail, le financement et l'investissement, sans oublier la gestion d'actifs. Lors du premier trimestre 2026, son bilan consolidé était évalué à 1,627 trillion d'euros, englobant 475 milliards d'euros en prêts à la clientèle et 626 milliards d'euros en dépôts [1]. Ces volumes reflètent l'envergure des transactions que la banque gère chaque jour, notamment dans son secteur de banque de détail, responsable des paiements sur internet, des transferts internationaux et des transactions sur mobile. Avec la progression de la digitalisation des services bancaires, la Société Générale, à l'instar de tout le secteur, est contrainte de renforcer ses mesures de prévention contre la fraude afin de maintenir la confiance de ses clients et de se conformer aux normes réglementaires et de cybersécurité qui reflètent ses principes de fiabilité et de sécurité.

C'est dans cette optique que le projet **SG SecureBank** a vu le jour : une application web conçue pour répondre à cette demande, en utilisant des modèles d'intelligence artificielle afin d'identifier de manière automatique les transactions suspectes.

1.1.2 Problématique

En raison de l'escalade de la numérisation des services bancaires, le volume de transactions que les banques gèrent tous les jours rend l'identification manuelle des fraudes inefficace. Les activités à l'échelle mondiale et mobile entraînent des dangers supplémentaires qui requièrent une vigilance constante. Le projet SG SecureBank a pour but d'établir un dispositif intelligent fondé sur l'intelligence artificielle qui permettra de détecter automatiquement les transactions suspectes et d'aider les analystes dans leur processus décisionnel.

1.1.3 Opportunité de marché

L'élaboration d'un système de détection de fraude qui utilise l'intelligence artificielle constitue une opportunité stratégique pour améliorer la sûreté dans le secteur bancaire. Cette méthode aide à minimiser les pertes économiques, à rehausser l'expérience des clients et à renforcer la confiance dans les prestations numériques. Elle satisfait aussi à l'exigence grandissante du secteur financier en termes de cybersécurité et d'automatisation.

1.1.4 Besoin identifié

Le but du projet est de créer une application web qui utilise des modèles d'apprentissage automatique pour détecter en temps réel les transactions suspectes. La solution devrait émettre des notifications aux analystes, favoriser la supervision opérationnelle et assurer le respect des normes réglementaires, y compris le RGPD.

1.1.5 Objectifs stratégiques du projet

Les principaux objectifs stratégiques du projet sont :

- Détecter automatiquement les transactions frauduleuses grâce aux modèles d'intelligence artificielle.
- Mettre en place un tableau de bord interactif avec des alertes en temps réel.
- Permettre la validation ou le blocage des transactions suspectes.
- Garantir la sécurité et la conformité des données selon le RGPD.
- Générer des rapports automatisés pour assurer le suivi et la traçabilité des fraudes.

1.1.6 Analyse SWOT

Afin d'évaluer la pertinence stratégique du projet SG SecureBank, une analyse SWOT a été réalisée (voir tableau 1.1).

L'analyse SWOT met en relief les points forts du projet qui associe l'analyse de données alimentées par l'intelligence artificielle à l'automatisation afin de détecter les fraudes. Cependant, la qualité des informations et le respect des normes réglementaires continuent d'être des facteurs déterminants pour garantir le bon fonctionnement du système. L'évolution constante des menaces nécessite aussi une actualisation régulière des modèles utilisés.

TABLE 1.1 – Analyse SWOT du projet SG SecureBank

Forces	Faiblesses
- Détection automatique des fraudes. - Comparaison de plusieurs modèles Machine Learning. - Détection en temps réel. - Tableau de bord interactif.	- Coût initial élevé de mise en oeuvre. - Dépendance à la qualité des données. - Risque de faux positifs. - Complexité technique.
Opportunités	Menaces
- Croissance des besoins en cybersécurité. - Extension vers d'autres services bancaires. - Amélioration continue des modèles IA. - Renforcement de la confiance client.	- Évolution des techniques de fraude. - Cyberattaques ciblant les systèmes bancaires. - Contraintes réglementaires RGPD. - Coût élevé de maintenance.

1.2 Analyse des besoins

1.2.1 Méthodologie de recueil des besoins

Les exigences du projet ont été définies sur la base du cahier des charges, enrichi par une approche axée sur l'utilisateur. Sans la présence d'un client véritable, les techniques mises en oeuvre (personas, simulations d'entretiens, questionnaires de priorité et études de processus d'affaires) ont été reconstituées pour simuler une approche professionnelle de la collecte des exigences.

Cette étude a aidé à déterminer les utilisateurs primaires du système, y compris l'analyste anti-fraude qui supervise les opérations douteuses, ainsi que le gestionnaire technique et conformité, qui est en charge de la sécurité, de l'implémentation du système et de la conformité au RGPD.

Les besoins fonctionnels identifiés sont les suivants :

- Détection automatique des transactions à risque.
- Génération d'alertes en temps réel.
- Visualisation des informations via un tableau de bord interactif.
- Validation ou blocage des transactions suspectes.
- Génération de rapports de suivi et de traçabilité.

Cette démarche a permis de définir une solution adaptée aux contraintes métier, techniques et réglementaires du secteur bancaire.

1.2.2 Analyse du processus métier

L'examen du processus opérationnel a été effectué en se basant sur les exigences du cahier des charges et les méthodes généralement employées dans l'industrie bancaire. Elle a facilité l'identification des phases clés du processus de gestion d'une transaction suspecte, les intervenants concernés, en plus des fonctionnalités escomptées de la solution SG SecureBank.

La procédure commence par la saisie d'une opération bancaire, qui sera par la suite examinée par le modèle d'apprentissage automatique. Ce système attribue un indice de risque qui permet de repérer les transactions susceptibles d'être frauduleuses. Dès qu'une limite de risque est franchie, une alerte est automatiquement déclenchée et dirigée vers l'analyste de fraude.

L'examineur analyse les données accessibles à partir du tableau de bord, puis opte pour valider la fraude ou pour catégoriser l'alerte en tant que faux positif. La décision est consignée pour garantir une traçabilité intégrale du processus et simplifier la gestion des alertes. Les personas, les cas d'utilisation et les documents détaillés dérivant de cette étude sont fournis en Annexe A.

1.2.3 Résultats de l'analyse des besoins

L'analyse des besoins a permis d'identifier les deux principaux profils utilisateurs de la solution :

- **Analyste fraude** : utilisateur chargé d'analyser les alertes, d'évaluer les transactions suspectes et de prendre les décisions de validation ou de rejet.
- **Responsable technique et conformité** : responsable de la sécurité du système, de son intégration dans l'environnement informatique de la banque et du respect des exigences réglementaires (RGPD).

Les besoins exprimés par ces utilisateurs ont conduit à identifier les fonctionnalités essentielles suivantes :

- Réduire le temps de traitement des alertes grâce à une plateforme centralisée.
- Assurer le suivi et la traçabilité des décisions prises sur les transactions suspectes.
- Le modèle de Machine Learning aide les analystes à détecter les fraudes, mais la décision finale de valider ou de bloquer une transaction est prise par un analyste humain.

Le système se fonde sur une méthode *Human-in-the-Loop* où un modèle Random Forest évalue automatiquement un indice de risque pour chaque transaction. Cependant, le dernier mot revient toujours à l'analyste fraude.

Chaque alerte peut évoluer selon les statuts suivants :

- **EN_COURS** : analyse en cours par l'analyste ;
- **CONFIRMÉ** : fraude confirmée après validation humaine ;
- **FAUX_POSITIF** : alerte considérée comme légitime après vérification ;
- **RÉSOLU** : traitement terminé avec décision enregistrée.

Cette structure assure une administration sûre et un suivi total des alertes, tout en unissant les capacités de l'intelligence artificielle à l'expertise humaine.

1.2.4 Recensement des besoins

L'évaluation des besoins a conduit à déterminer les exigences clés indispensables pour le développement de la plateforme SG SecureBank. Trois catégories regroupent ces besoins : fonctionnels, techniques et associés aux données.

1.2.4.1 Besoins fonctionnels

Les fonctionnalités principales attendues sont :

- Détection automatique des transactions suspectes grâce aux modèles de Machine Learning.
- Calcul et affichage d'un score de risque pour chaque transaction.
- Gestion des alertes par les analystes fraude (validation, rejet, résolution).

- Blocage d'un compte en cas de fraude confirmée.
- Visualisation des indicateurs via un tableau de bord interactif.
- Génération de rapports assurant le suivi et la traçabilité des actions.
- Gestion des utilisateurs selon leurs rôles et leurs droits d'accès.

1.2.4.2 Besoins techniques

Les besoins techniques concernent l'architecture et la sécurité de la solution :

- Développement d'un back-end avec Flask connecté à une base Oracle XE.
- Intégration de modèles de Machine Learning tels que Random Forest, Decision Tree, ANN et Logistic Regression.
- Mise en place d'une authentification sécurisée avec JWT.
- Garantie de bonnes performances pour permettre la détection en quasi temps réel.
- Compatibilité de l'application avec les principaux navigateurs.

1.2.4.3 Besoins liés aux données

Les modèles de détection nécessitent des données fiables et représentatives. Les principales exigences sont :

- Utilisation d'un jeu de données transactionnel adapté à la détection de fraude.
- Nettoyage et préparation des données (suppression des anomalies, normalisation et encodage).
- Gestion du déséquilibre entre transactions normales et frauduleuses.
- Respect des exigences du RGPD et garantie de la traçabilité des données utilisées.

1.2.5 Synthèse des besoins

Les exigences identifiées conduisent à l'élaboration d'une solution fusionnant intelligence artificielle, sécurité et contrôle humain. Il est donc nécessaire que le système SG SecureBank automatise l'identification des fraudes tout en maintenant une approbation humaine pour les décisions critiques.

1.2.6 Priorisation des besoins (méthode MoSCoW)

Afin de garantir la livraison des fonctionnalités essentielles dans le calendrier académique imparti, les besoins identifiés ont été priorisés selon la méthode MoSCoW (Must Have, Should Have, Could Have, Won't Have). Le tableau 1.2 présente la répartition des fonctionnalités selon leur niveau de priorité.

1.2.7 Veille technologique

Avant de choisir les technologies à utiliser pour la création de SG SecureBank, une analyse comparative a été effectuée pour déterminer les options les plus appropriées aux exigences du projet.

Cette surveillance s'est concentrée sur trois axes majeurs : les Frameworks côté serveur, les technologies côté client et les systèmes de gestion de bases de données. Les critères pris en compte comprennent l'intégration aisée avec les modèles de Machine Learning, les performances, la sûreté, la facilité d'entretien et l'ajustabilité au milieu bancaire.

TABLE 1.2 – Priorisation des besoins selon la méthode MoSCoW

Priorité	Besoins concernés
Must Have	Authentification sécurisée (JWT) ; Détection automatique des transactions suspectes (Machine Learning) ; Calcul et affichage du score de risque ; Gestion des alertes (validation, rejet, résolution) ; Tableau de bord interactif.
Should Have	Blocage d'un compte en cas de fraude confirmée ; Génération de rapports et journalisation (traçabilité) ; Gestion des rôles utilisateurs (administrateur, analyste fraude, responsable conformité).
Could Have	Modification du profil utilisateur via l'interface graphique ; Affichage des messages d'erreur directement dans les formulaires ; Intégration d'un plugin d'accessibilité tiers.
Won't Have	Connexion à un système bancaire réel ; Traitement de millions de transactions en temps réel ; Application mobile Android/iOS ; Intégration avec des services de paiement externes ; Notifications automatiques par SMS ou e-mail ; Système complet d'explicabilité des modèles (XAI) ; Déploiement industriel avec infrastructure distribuée.

1.2.7.1 Choix du framework back-end

Nous avons examiné divers frameworks Python : Flask, Django et FastAPI.

Flask a été choisi en raison de sa légèreté, de sa souplesse et de sa capacité à s'intégrer aisément avec des bibliothèques de Data Science comme scikit-learn. Cela permet aussi un développement rapide et ajusté aux exigences du projet.

1.2.7.2 Choix de la technologie front-end

Une comparaison a été effectuée entre les solutions Bootstrap, Tailwind CSS et Material UI.

La conception de l'interface utilisateur a été confiée à Bootstrap en raison de ses composants préexistants, de sa compatibilité avec HTML/CSS/JavaScript et de sa capacité à produire une interface responsive adaptée aux analystes de fraude.

1.2.7.3 Choix du système de gestion de base de données

L'étude a porté sur Oracle Database XE, PostgreSQL ainsi que MySQL.

Nous avons opté pour Oracle Database XE en raison de sa solidité, de l'adhésion à des propriétés ACID, de ses capacités transactionnelles et de son usage commun dans les contextes professionnels bancaires.

1.2.8 Analyse des risques

Une évaluation des risques a été effectuée dans le but de repérer les facteurs susceptibles de nuire au succès du projet. Les risques majeurs incluent :

- le déséquilibre des données pouvant influencer les performances des modèles ;

- les faux positifs pouvant augmenter la charge des analystes ;
- les contraintes de temps liées au calendrier académique ;
- les problèmes de sécurité liés aux données bancaires ;
- les incompatibilités entre les dépendances techniques.

Des actions de prévention ont été mises en place, y compris l'amélioration de la préparation des données, l'évaluation fréquente des modèles, la protection des accès et la gestion des versions des bibliothèques.

Les risques identifiés ont été classés selon leur probabilité d'occurrence et leur impact potentiel sur le projet, afin de prioriser les actions de prévention. Le tableau 1.3 présente la matrice des risques retenue ainsi que les principales mesures préventives et correctives mises en place.

TABLE 1.3 – Matrice des risques du projet SG SecureBank

Risque	Probabilité	Impact	Action préventive/corrective
Déséquilibre des données (transactions normales vs frauduleuses)	Élevée	Élevé	Techniques de rééquilibrage, tests de validation multiples
Contraintes de temps liées au calendrier académique	Élevée	Élevé	Priorisation MoSCoW, suivi via diagramme de Gantt
Faux positifs augmentant la charge des analystes	Moyenne	Moyen	Ajustement du seuil de score de risque, comparaison de plusieurs modèles
Incompatibilités entre dépendances techniques (Flask, Oracle XE, bibliothèques Python)	Moyenne	Faible	Environnement virtuel dédié, fichier <code>requirements.txt</code> versionné
Problèmes de sécurité liés aux données bancaires	Faible	Élevé	Authentification JWT, hachage des mots de passe, tests de sécurité (<code>test_security.py</code>)

Les risques combinant une probabilité et un impact élevés (déséquilibre des données, contraintes de calendrier) ont fait l'objet d'un traitement prioritaire dès les premières phases du projet.

1.2.9 Impacts environnementaux et sociaux

L'élaboration d'un système de détection de fraude reposant sur l'intelligence artificielle peut entraîner des conséquences environnementales inhérentes aux ressources requises pour la formation et l'implémentation des modèles.

L'usage de modèles traditionnels de Machine Learning tels que Random Forest, Decision Tree et Logistic Regression favorise la réduction de la consommation d'énergie comparé à des modèles de Deep Learning plus sophistiqués.

Sur le plan social, la méthode *Human-in-the-Loop* assure un contrôle humain dans les décisions délicates. Le modèle attribue une note de risque, néanmoins, la décision ultime demeure entre les mains de l'analyste fraude.

1.2.10 Évaluation globale de la faisabilité

L'évaluation de la faisabilité du projet SG SecureBank a été menée selon sept dimensions, conformément aux exigences du cahier des charges.

Contraintes techniques : Le projet s'appuie sur des technologies open source (Flask, Oracle Database XE, Scikit-learn) maîtrisées dans un cadre académique, réduisant les risques d'incompatibilité et de coûts de licence.

Contraintes légales : La solution manipule des données bancaires sensibles, imposant le respect du RGPD dès la conception (minimisation des données, traçabilité des traitements, droit à la suppression des comptes).

Contraintes de sécurité : L'authentification JWT, le hachage des mots de passe et la journalisation des actions sensibles répondent aux exigences de sécurité propres au secteur bancaire .

Contraintes d'accessibilité : Les critères WCAG AA (contrastes, attributs ARIA, labels de formulaires) ont été intégrés dès la conception des interfaces, avec certaines limites assumées (absence de plugin d'accessibilité tiers.).

Typologie et qualité des données : Le projet repose sur un jeu de données simulé de 1 050 clients, 1 487 comptes et 5 205 transactions, soumis à des contrôles de qualité (doublons, valeurs manquantes, cohérence des formats) avant utilisation par les modèles de Machine Learning .

Contraintes financières : L'utilisation de technologies open source limite le coût du prototype académique à l'hébergement serveur (10 à 50 /mois) et au nom de domaine (10 à 15 /an), un budget professionnel plus conséquent étant nécessaire pour une mise en production réelle .

Délais de réalisation : Le projet est structuré sur huit mois (janvier à août 2026) , avec une priorisation MoSCoW des fonctionnalités permettant d'absorber d'éventuels retards sans compromettre la livraison des fonctionnalités essentielles.

Au regard de ces sept dimensions, le projet SG SecureBank apparaît réalisable dans le cadre académique imparti. Une transition vers un contexte bancaire réel nécessiterait toutefois un renforcement des exigences de sécurité, de disponibilité et de conformité, en particulier sur la distinction des rôles utilisateurs .

1.3 Présentation du cahier des charges

Le document de référence pour le projet SG SecureBank est le cahier des charges. Ce document détaille les spécifications fonctionnelles, techniques et organisationnelles requises pour concevoir et développer la solution.

Congu en prenant en compte l'examen des besoins, la surveillance technologique et l'évaluation de la faisabilité, ce document précise les critères que la plateforme doit satisfaire, tout en tenant compte des restrictions organisationnelles, techniques et réglementaires inhérentes au secteur bancaire.

Le projet SG SecureBank s'attèle à la création d'une application web intelligente visant à identifier les transactions bancaires suspectes, en faisant appel à des modèles de Machine Learning. La solution a pour objectif d'aider les analystes de la fraude, d'accélérer le traitement des alertes et de faciliter la prise de décision en calculant un indice de risque.

Le cahier des charges décrit notamment :

- le périmètre fonctionnel de la solution .
- les fonctionnalités du front-office et du back-office .
- l'architecture technique et les technologies utilisées .
- la solution d'hébergement et la stratégie de sauvegarde .

- les parties prenantes et l'organisation du projet .
- la planification, les ressources nécessaires, le budget et les risques associés.

Ce document garantit la correspondance entre les exigences détectées, les décisions techniques prises et les buts du projet, tout en agissant comme un outil de communication entre les différents participants engagés.

1.3.1 Périmètre du projet

Les frontières fonctionnelles et techniques de la solution élaborée sont déterminées par l'étendue du projet SG SecureBank. Cela permet de déterminer les fonctionnalités incluses et celles omises, assurant ainsi un meilleur contrôle des échéances, des ressources et des restrictions du projet éducatif.

SG SecureBank est un projet visant à développer un portail web intelligent pour aider les analystes de fraude à superviser les opérations bancaires. La solution met en oeuvre des modèles d'apprentissage automatique pour identifier les actions suspectes et évaluer un score de risque, ceci afin d'assister les utilisateurs dans leur processus décisionnel.

1.3.1.1 Fonctionnalités incluses

La plateforme intègre les fonctionnalités principales suivantes :

- Authentification sécurisée des utilisateurs avec JWT et gestion des rôles (administrateur, analyste fraude et responsable conformité).
- Consultation des clients, comptes et transactions bancaires.
- Consultation et analyse des transactions.
- Détection automatique des transactions suspectes grâce aux modèles de Machine Learning.
- Calcul d'un score de risque pour chaque transaction.
- Génération et gestion des alertes de fraude.
- Validation, classification et clôture des alertes par l'analyste fraude.
- Blocage d'un compte en cas d'activité frauduleuse confirmée.
- Tableau de bord interactif présentant les indicateurs de suivi et statistiques.
- Génération de rapports et journalisation des actions afin d'assurer la traçabilité.

1.3.1.2 Fonctionnalités exclues

Afin de respecter les contraintes du projet, certaines fonctionnalités ne sont pas prises en charge dans cette première version :

- Connexion à un système bancaire réel.
- Traitement de millions de transactions en temps réel.
- Application mobile Android ou iOS.
- Intégration avec des services de paiement externes.
- Notifications automatiques par SMS ou courrier électronique.
- Système complet d'explicabilité des modèles (XAI).
- Déploiement industriel avec infrastructure distribuée.

1.3.1.3 Objectifs fonctionnels

Les buts primordiaux du projet visent à diminuer la durée d'examen des opérations suspectes, à regrouper les informations essentielles pour les experts anti-fraude, et à optimiser la qualité des décisions par le biais du calcul automatique d'une note de risque.

Cette solution utilise une stratégie dite Human-in-the-Loop , où le modèle d'apprentissage automatique soutient l'analyste sans pour autant supplanter la validation des décisions critiques par l'homme.

1.3.1.4 Contraintes du périmètre

Le développement de SG SecureBank respecte plusieurs contraintes :

- Respect du calendrier académique.
- Utilisation de Flask pour le back-end et HTML/CSS/JavaScript/Bootstrap pour le front-end.
- Utilisation d'Oracle Database XE pour la gestion des données.
- Utilisation des modèles Machine Learning développés avec Scikit-learn.
- Respect des exigences du RGPD concernant les données personnelles.
- Compatibilité avec les navigateurs web principaux et conception responsive.

En conséquence, l'étendue déterminée favorise l'accent sur les fonctionnalités fondamentales de détection et de gestion des fraudes, tout en assurant la réalisation du projet dans le calendrier programmé.

1.3.2 Description fonctionnelle de la solution

SG SecureBank est un outil en ligne intelligent conçu pour aider les analystes de fraude dans l'identification, l'examen et le traitement des transactions bancaires suspectes. La solution intègre une interface web, une API de back-end qui a été élaborée avec Flask, une base de données Oracle XE, ainsi qu'un module de Machine Learning qui attribue un score de risque à chaque transaction.

Le but principal est de rendre automatique le repérage des transactions suspectes, tout en gardant une vérification humaine pour les décisions cruciales, en suivant une approche *Human-in-the-Loop*.

1.3.2.1 Module d'authentification

Ce module garantit la protection des accès à la plateforme en se basant sur une authentification via JWT (JSON Web Token). Ce système facilite la gestion des sessions et la régulation des permissions selon les rôles des utilisateurs :

- **Analyste fraude** : analyse les transactions suspectes et traite les alertes.
- **Responsable conformité** : consulte les indicateurs et les rapports.
- **Administrateur** : assure la gestion technique de la plateforme.

1.3.2.2 Module de gestion des clients, comptes et transactions

Ce dispositif offre aux utilisateurs habilités la possibilité d'accéder aux renseignements indispensables pour l'analyse des transactions bancaires. Les fonctionnalités principales sont :

- consultation des clients et des comptes associés .
- création d'une nouvelle transaction via un formulaire dédié .
- consultation de l'historique des transactions, avec pour chacune le montant, le type, les pays d'origine et de destination, ainsi que le score de risque calculé.

1.3.2.3 Module de détection intelligente de fraude

Ce module constitue le noyau central de la solution. Il se sert de modèles d'apprentissage automatique pour examiner les transactions et déceler les agissements douteux.

Voici les étapes fondamentales du processus :

- nettoyage et préparation des données .
- transformation et encodage des variables .
- normalisation des données .
- application du modèle de prédiction.

Plusieurs modèles ont été étudiés :

- Régression Logistique .
- Arbre de Décision (*Decision Tree*) .
- Forêt Aléatoire (*Random Forest*) .
- Réseau de neurones artificiels (*MLPClassifier*).

Le modèle sélectionné détermine un indice de risque pour détecter les transactions susceptibles d'être frauduleuses.

1.3.2.4 Module de gestion des alertes

Quand une transaction excède un niveau de risque prédéfini, une notification est automatiquement créée. L'analyste en détection de fraude a donc la possibilité de revoir et d'ajuster son statut.

Les différentes configurations possibles sont :

- **EN_COURS** : analyse en cours ;
- **CONFIRMÉ** : fraude validée par l'analyste ;
- **FAUX_POSITIF** : transaction considérée comme légitime ;
- **RÉSOLU** : traitement terminé.

Cette organisation assure un suivi des décisions tout en préservant une surveillance humaine.

1.3.2.5 Tableau de bord de supervision

Le tableau de bord fournit une vue globale de l'activité de détection de fraude. Il présente :

- le nombre de transactions analysées .
- le nombre d'alertes générées .
- les fraudes confirmées .
- les faux positifs .
- la répartition des niveaux de risque.

Avec ces outils, SG SecureBank facilite le contrôle renforcé des transactions, raccourcit la durée de traitement des alertes et assiste les analystes dans leur prise de décisions.

1.4 Architecture technique et choix technologiques

1.4.1 Architecture générale de la solution

SG SecureBank est construit sur une structure en trois couches qui permet de distinguer l'interface utilisateur, la logique d'entreprise et la gestion des données.

Elle est composée de trois couches principales :

- **Couche présentation (Front-end)** : interface web permettant aux analystes fraude d'interagir avec la plateforme.
- **Couche applicative (Back-end)** : API développée avec Flask assurant la gestion métier, l'authentification et la communication avec le modèle Machine Learning.

- **Couche données** : base Oracle XE permettant le stockage sécurisé des clients, comptes, transactions et alertes.

Le fonctionnement général est basé sur le flux suivant :

Utilisateur → Interface Web → API Flask → Base Oracle XE

Le module de détection de fraude intervient lors de l'analyse des transactions :

Transaction → Prétraitement → Modèle ML → Score de risque → Alerte

1.4.2 Technologies utilisées

Les décisions concernant les technologies ont été prises en tenant compte de plusieurs facteurs : la facilité de développement, l'efficacité, la sécurité et la correspondance avec les exigences du projet.

1.4.2.1 Technologies Front-end

L'interface utilisateur est développée avec les technologies suivantes :

- **HTML5** : utilisé pour structurer les différentes pages de l'application (connexion, tableau de bord, transactions et alertes).
- **CSS3** : utilisé pour la mise en forme et l'amélioration de l'expérience utilisateur.
- **JavaScript** : permet la gestion des interactions dynamiques et la communication avec l'API Flask.
- **Bootstrap** : choisi pour faciliter la création d'une interface responsive, ergonomique et compatible avec les standards d'accessibilité web.

1.4.2.2 Technologies Back-end

Le back-end repose sur :

- **Python** : choisi pour sa compatibilité avec les outils de Data Science et Machine Learning.
- **Flask** : framework utilisé pour développer l'API REST, gérer les utilisateurs, les transactions et les alertes.

Les principales API développées sont :

TABLE 1.4 – Principales API de SG SecureBank

API	Fonction
/api/login	Authentification utilisateur
/api/transactions	Consultation et création des transactions
/api/transactions/<id>	Consultation d'une transaction spécifique

1.4.2.3 Base de données

La solution utilise **Oracle Database XE** pour stocker les données du système.

Ce choix est justifié par :

- sa robustesse pour les applications transactionnelles .
- sa conformité aux propriétés ACID .
- son utilisation fréquente dans le secteur bancaire .
- sa disponibilité gratuite dans un contexte académique.

Les principales tables sont :

- **CLIENTS** : informations des clients .
- **COMPTES** : données des comptes bancaires .
- **TRANSACTIONS** : historique des opérations .
- **FRAUD** : alertes générées .
- **USERS** : gestion des utilisateurs.

1.4.2.4 Technologies Machine Learning

Le module intelligent repose sur plusieurs modèles étudiés :

- Régression Logistique .
- Arbre de Décision .
- Random Forest .
- ANN.

Suite à l'évaluation des performances, le modèle sélectionné est incorporé dans l'application pour permettre un calcul automatique du score de risque pour chaque transaction.

Le processus de détection comprend :

1. Préparation des données .
2. Application du modèle .
3. Calcul du niveau de risque .
4. Génération d'une alerte si nécessaire.

1.4.3 Sécurité de la solution

La sécurité de SG SecureBank repose sur plusieurs mécanismes :

- **Authentification JWT** pour sécuriser les échanges entre le front-end et le back-end.
- **Gestion des rôles** afin de limiter l'accès aux fonctionnalités selon le profil utilisateur.
- **Hachage des mots de passe** pour protéger les informations d'identification.
- **Journalisation des actions sensibles** afin d'assurer la traçabilité des opérations.

Ces dispositions assurent la protection, la préservation et la sûreté des informations bancaires gérées par la plateforme.

1.5 Solution d'hébergement et nom de domaine préconisé

1.5.1 Choix de la solution d'hébergement

La plateforme SG SecureBank doit être hébergée en garantissant disponibilité, sécurité et performances requis pour gérer les données bancaires et les prédictions du modèle d'apprentissage automatique.

Pour ce projet, il est conseillé d'opter pour une solution d'hébergement en cloud afin de jouir d'une infrastructure adaptable et modulable. Des solutions comme **Microsoft Azure**, **Amazon Web Services (AWS)** ou **Google Cloud Platform (GCP)** faciliteraient l'ajustement des ressources en fonction du volume de transactions géré.

Les principaux critères de choix sont :

- **Performance** : possibilité d'augmenter les ressources serveur selon les besoins de l'application.
- **Disponibilité** : garantie d'un accès continu à la plateforme grâce aux mécanismes de haute disponibilité proposés par les fournisseurs cloud.
- **Sécurité** : présence de mécanismes de protection des données, de contrôle d'accès et de surveillance des infrastructures.
- **Coût maîtrisé** : paiement selon l'utilisation réelle des ressources, permettant de limiter les dépenses lors des phases de développement et de test.

1.5.2 Architecture d'hébergement proposée

L'architecture d'hébergement recommandée est composée de plusieurs services :

- **Serveur applicatif** : hébergement de l'API Flask et du module de détection de fraude.
- **Serveur de base de données** : hébergement sécurisé de la base Oracle contenant les informations clients, comptes et transactions.
- **Stockage sécurisé** : conservation des sauvegardes et des fichiers nécessaires au fonctionnement de la plateforme.
- **Certificat SSL/TLS** : chiffrement des communications entre les utilisateurs et l'application.

1.5.3 Nom de domaine proposé

Afin de faciliter l'accès à la plateforme, un nom de domaine professionnel pourrait être utilisé :

www.sg-securebank.com

Ce nom permettrait d'identifier clairement le service et de renforcer l'image professionnelle de la solution.

1.5.4 Justification du choix

L'utilisation d'une infrastructure cloud constitue une option appropriée pour répondre aux exigences de SG SecureBank. Elle fournit une harmonie entre efficacité, sûreté et prix, tout en garantissant une expansion future de la plateforme selon la croissance du volume de transactions et les exigences business.

Pour le projet universitaire, l'application est capable de fonctionner soit sur un environnement local, soit sur une solution cloud gratuite dotée de ressources limitées. Par contre, une mise en oeuvre effective dans un cadre bancaire exigerait une structure professionnelle dotée de critères de sécurité et de disponibilité élevés. Il conviendra de sélectionner une région d'hébergement située dans l'Union Européenne (par exemple Europe de l'Ouest) chez le fournisseur cloud retenu, afin de garantir la conformité RGPD et d'éviter tout transfert de données bancaires hors de l'UE.

1.6 Stratégie de sauvegarde

Sur la plateforme SG SecureBank, la sécurité des données est primordiale, étant donné que le système gère des données délicates touchant les clients, les comptes, les opérations et les signalements de fraude.

Nous avons mis en place une stratégie de sauvegarde personnalisée pour assurer la disponibilité de nos données et minimiser les risques de perte en cas d'incident technique, d'erreur de l'utilisateur ou de cyberattaque.

1.6.1 Types de sauvegarde

La stratégie retenue repose sur plusieurs niveaux de sauvegarde :

- **Sauvegarde complète périodique** : une copie complète de la base de données Oracle XE est réalisée régulièrement afin de pouvoir restaurer l'ensemble des informations du système.
- **Sauvegarde incrémentielle** : seules les modifications effectuées depuis la dernière sauvegarde sont enregistrées afin de réduire le temps et l'espace de stockage nécessaires.
- **Sauvegarde du code source** : le code de l'application Flask, les fichiers front-end et les configurations sont conservés dans un dépôt Git afin d'assurer le suivi des versions.

1.6.2 Stockage et sécurité des sauvegardes

Il est impératif de conserver les sauvegardes dans un espace sûr et distinct de l'environnement de production pour minimiser les chances de perte de données en même temps.

Les principales mesures de protection sont :

- chiffrement des fichiers de sauvegarde .
- contrôle des accès aux sauvegardes .
- conservation de plusieurs versions historiques .
- vérification régulière de la restauration des données.

1.6.3 Plan de restauration

En cas d'incident, un processus de restauration est prévu :

1. Identification de la cause de l'incident .
2. Récupération de la dernière sauvegarde valide .
3. Restauration de la base de données et des composants applicatifs .
4. Vérification du bon fonctionnement de la plateforme.

Cette approche garantit la permanence du service, la protection des informations bancaires et la crédibilité de la solution SG SecureBank.

1.7 Parties prenantes et gestion de projet

1.7.1 Identification des parties prenantes

La réalisation du projet SG SecureBank implique plusieurs acteurs ayant des rôles et des responsabilités différentes.

Les principales parties prenantes sont :

- **Chef de projet** : assure la planification, le suivi de l'avancement, la coordination des tâches et la gestion des risques.
- **Développeur Full Stack** : responsable de la conception et du développement de l'application web, du back-end Flask et de l'intégration des différents modules.

- **Data Scientist / Machine Learning Engineer** : chargé de la préparation des données, de l'entraînement des modèles et de l'évaluation des performances.
- **Analyste fraude (utilisateur final)** : utilise la plateforme pour analyser les alertes et valider les décisions liées aux transactions suspectes.
- **Responsable conformité et sécurité** : veille au respect des exigences réglementaires, notamment le RGPD, et contrôle les aspects liés à la sécurité des données.

1.7.2 Méthode de gestion de projet

Le projet étant mené par une seule personne, une approche agile simplifiée de type Kanban personnel a été adoptée, plutôt qu'un Scrum classique qui suppose une répartition de rôles entre plusieurs membres d'une équipe (Scrum Master, Product Owner, équipe de développement).

Cette organisation permet :

- un suivi régulier de l'avancement .
- une adaptation progressive aux besoins identifiés .
- une priorisation des fonctionnalités essentielles .
- une meilleure gestion des risques techniques.

Le projet est organisé en plusieurs phases :

1. Analyse des besoins et définition du périmètre .
2. Conception de l'architecture technique .
3. Préparation des données et développement du modèle Machine Learning .
4. Développement de l'application web .
5. Tests, corrections et validation finale.

Ce système assure un suivi approfondi du projet, le respect des échéances programmées et la fourniture d'une solution alignée sur les objectifs établis.

1.8 Planification du projet, ressources humaines et budget

1.8.1 Rétroplanning du projet

Le projet SG SecureBank a été lancé en janvier 2026 et doit se poursuivre jusqu'à sa présentation prévue en septembre 2026. Le projet a été structuré en différentes étapes afin d'assurer un déroulement progressif des activités, depuis l'analyse des besoins jusqu'à la préparation de la soutenance.

Le tableau 1.5 présente le rétroplanning prévisionnel du projet, mettant en évidence les principales phases de réalisation ainsi que leur répartition sur la durée du projet.

Le diagramme de Gantt ci-après affiche le calendrier projeté : L'organisation permet de distribuer les diverses tâches dans le temps et de repérer les retards possibles qui pourraient affecter la date de présentation.

1.8.2 Ressources humaines nécessaires

Pour ce projet académique, l'équipe déployée est essentiellement constituée d'un chef de projet développeur responsable de l'élaboration et de la mise en oeuvre de la solution.

Les principales ressources nécessaires sont :

TABLE 1.5 – Planning prévisionnel du projet SG SecureBank

Activités	Jan	Fév	Mars	Avril	Mai	Juin	Juil	Août
Analyse des besoins et cahier des charges	X	X						
Veille technologique et choix des solutions	X	X						
Conception de l'architecture et base de données		X	X					
Préparation et traitement des données			X	X				
Développement du modèle Machine Learning				X	X			
Développement de l'application web Flask					X	X		
Intégration front-end et tests fonctionnels						X	X	
Optimisation, documentation et corrections							X	X
Préparation du rapport et soutenance								X

- **Chef de projet / Développeur Data** : responsable de l'analyse des besoins, de la conception de l'architecture, du développement de l'application et de l'intégration des modèles Machine Learning.
- **Data Scientist** : chargé de la préparation des données, de l'entraînement des modèles, de leur évaluation et de l'amélioration des performances.
- **Développeur Web** : responsable de la conception de l'interface utilisateur, de l'intégration front-end et de la communication avec l'API Flask.
- **Encadrant pédagogique** : assure le suivi du projet, valide les choix techniques et accompagne la réalisation du livrable final.

1.8.3 Estimation du budget

Le budget du projet prend en compte les ressources humaines, l'infrastructure technique et les outils nécessaires au développement. Le tableau 1.6 présente une estimation des principaux postes de dépenses du projet SG SecureBank.

L'emploi de technologies open source permet de conserver un coût abordable pour le prototype. Dans une situation bancaire concrète, un budget plus conséquent serait requis pour l'hébergement sûr, les vérifications de sécurité, l'entretien et les équipes techniques réservées.

TABLE 1.6 – Estimation budgétaire du projet SG SecureBank

Poste	Estimation
Développement et conception	Réalisé dans le cadre académique (pas de coût salarial)
Technologies utilisées (Python, Flask, Bootstrap, scikit-learn)	0 (solutions open source)
Base de données Oracle XE	0 (version Express utilisée)
Hébergement serveur	Environ 10 à 50 /mois selon la solution choisie
Nom de domaine	Environ 10 à 15 /an
Maintenance et évolution	Variable selon les besoins futurs

1.9 Identification des goulots d'étranglement et des risques de retard

Plusieurs éléments pourraient causer des retards ou freiner le progrès du projet SG SecureBank durant sa mise en uvre. La détection précoce de ces obstacles permet d'établir des mesures préventives et correctives afin de respecter le calendrier initialement établi. Le tableau 1.7 présente les principaux goulots d'étranglement identifiés, leurs impacts potentiels ainsi que les actions préventives envisagées.

1.9.1 Stratégie d'anticipation

Pour minimiser les risques de retard, nous avons opté pour une stratégie progressive. La plateforme a été développée en suivant la méthode MoSCoW pour évaluer les fonctionnalités prioritaires, avec un accent sur l'implémentation des fonctionnalités essentielles en premier lieu :

- Authentification sécurisée des utilisateurs .
- Gestion des transactions .
- Détection automatique de fraude .
- Gestion des alertes .
- Tableau de bord de supervision.

Les fonctionnalités secondaires pourront être optimisées ou intégrées ultérieurement en fonction des limitations de temps et des moyens disponibles.

Cette structure permet d'anticiper les risques majeurs relatifs aux retards, aux limitations techniques et à la qualité des données, garantissant ainsi que le projet SG SecureBank sera livré dans les temps pour la soutenance de septembre 2026.

TABLE 1.7 – Identification des goulots d'étranglement du projet

Goulot d'étranglement	Impact potentiel	Actions préventives
Qualité et disponibilité des données	Difficulté d'obtenir un modèle Machine Learning performant en raison du déséquilibre entre transactions normales et frauduleuses.	Réaliser un nettoyage des données, appliquer des techniques de rééquilibrage et effectuer plusieurs tests de validation.
Optimisation du modèle de détection de fraude	Des performances insuffisantes du modèle peuvent retarder l'intégration dans l'application.	Comparer plusieurs modèles (Random Forest, Decision Tree, Régression Logistique, MLPClassifier) et sélectionner le plus adapté.
Problèmes techniques liés à l'intégration	Des incompatibilités entre Flask, Oracle XE et les bibliothèques Python peuvent ralentir le développement.	Utiliser un environnement virtuel, fixer les versions des dépendances dans le fichier <code>requirements.txt</code> et effectuer des tests réguliers.
Temps limité avant la soutenance	Un retard dans une phase peut impacter les étapes suivantes.	Prioriser les fonctionnalités essentielles (Must Have) et suivre régulièrement l'avancement selon le diagramme de Gantt.
Tests et correction des anomalies	La découverte tardive d'erreurs peut retarder la livraison finale.	Prévoir des phases de tests progressives tout au long du développement.

La conception et le développement d'une solution web

2.1	Architecture technique	27
2.1.1	Brique Data	28
2.1.2	Collecte et gouvernance des données	29
2.1.3	Justification des choix techniques	30
2.2	Maquettes et prototypes	31
2.2.1	Interfaces principales de l'application	31
2.2.1.1	Page de connexion	31
2.2.1.2	Tableau de bord	32
2.2.1.3	Page transactions	32
2.2.1.4	Choix graphiques et ergonomiques	33
2.2.2	Respect des critères ergonomiques	33
2.2.2.1	Auto-évaluation et ajustements	33
2.2.3	Présentation du développement front-end	34
2.2.3.1	Choix techniques et justification	34
2.2.3.2	Approche mobile-first	35
2.2.3.3	Tests réalisés sur plusieurs résolutions	35
2.2.3.4	Limites identifiées	35
2.2.4	La présentation du développement back-end	36
2.2.4.1	Standards de programmation et bonnes pratiques	36
2.2.4.2	Schéma relationnel	37
2.2.4.3	Optimisations des requêtes SQL	37
2.2.4.4	Endpoints sécurisés	38
2.2.4.5	Documentation des API	38
2.2.4.6	Mesures de performance	38
2.2.4.7	Limites identifiées	39
2.2.5	Tests et qualité logicielle	39
2.2.5.1	Stratégie de tests	39
2.2.5.2	Bug détecté et corrigé grâce aux tests	40
2.2.5.3	Correction d'un second bug	41
2.2.5.4	Limites identifiées	41
2.2.5.5	Limites des tests	41
2.2.6	Conformité au RGPD	42
2.2.7	Accessibilité	42
2.2.8	Livrables	42
2.2.9	Processus de gestion des mises à jour et des incidents post-déploiement	43
2.2.9.1	Plan de mise à jour	43

2.2.9.2	Processus de déploiement	43
2.2.9.3	Gestion des incidents et scénarios d'alertes	44
2.2.10	Retours des parties prenantes et ajustements	44
2.2.10.1	Retours sur l'organisation et la planification	45
2.2.10.2	Retours sur l'interface et l'ergonomie	45
2.2.10.3	Retours sur la sécurité et la conformité	45
2.2.10.4	Retours sur la conformité RGPD	45
2.2.10.5	Synthèse	45
A.1	Personas	47
A.1.1	Persona 1 : Analyste fraude	47
A.1.2	Persona 2 : Responsable conformité et sécurité	47
A.2	Compte-rendu d'atelier simulé	47
A.3	Cas d'utilisation principaux	48

2.1 Architecture technique

SG SecureBank est construit sur une structure web modulaire en trois niveaux qui distingue l'interface de l'utilisateur, la logique d'entreprise et le traitement des données. Cette structure facilite la gestion de la maintenance, améliore la sûreté et encourage l'évolution de l'application.

La première couche est associée au **Front-end**, réalisé avec HTML5, CSS3, Bootstrap 5 et JavaScript. Elle propose une interface web réactive qui donne aux utilisateurs la possibilité d'accéder aux principales fonctionnalités de la plateforme, y compris le tableau de bord, la gestion des clients, des comptes, des transactions et la vigilance face à la fraude. Les performances et les données statistiques sont présentées par le biais de graphiques interactifs créés à l'aide de **Chart.js**, ce qui enrichit l'affichage des informations et optimise l'expérience de l'utilisateur.

Le second niveau est le **Back-end**, mis en place en Python en utilisant le framework **Flask**. L'application adopte une structure modulaire fondée sur les `Blueprints`, ce qui facilite la division des divers modules en termes de fonctionnalité comme l'authentification, la gestion des clients, des comptes, des transactions, des alertes et du tableau de bord. Le fichier `fraud_detector.py` comprend les modèles de Machine Learning pour juger de manière automatique le degré de risque des transactions et produire un score de fraude.

La troisième couche abrite le **Oracle Database XE**, un système de gestion de base de données relationnelle garantissant la conservation des données concernant les utilisateurs, les clients, les comptes, les transactions et les alertes. Oracle assure la cohérence et l'intégrité des données en adhérant aux propriétés ACID et en appliquant des contraintes relationnelles.

L'échange de données au format JSON entre le Front-end et le Back-end s'effectue via une **API REST**. Cette structure facilite l'incorporation de nouveaux services, rend la maintenance de l'application plus aisée et encourage son adaptation à une infrastructure distribuée si le nombre de transactions croît.

L'architecture globale de la solution est illustrée dans la Figure 2.1.

Plusieurs dispositifs complémentaires garantissent la sécurité : l'authentification fondée sur les **JSON Web Tokens (JWT)**, le hachage sécurisé des mots de passe, la vérification des données entrées pour réduire les menaces d'injection SQL, ainsi que la

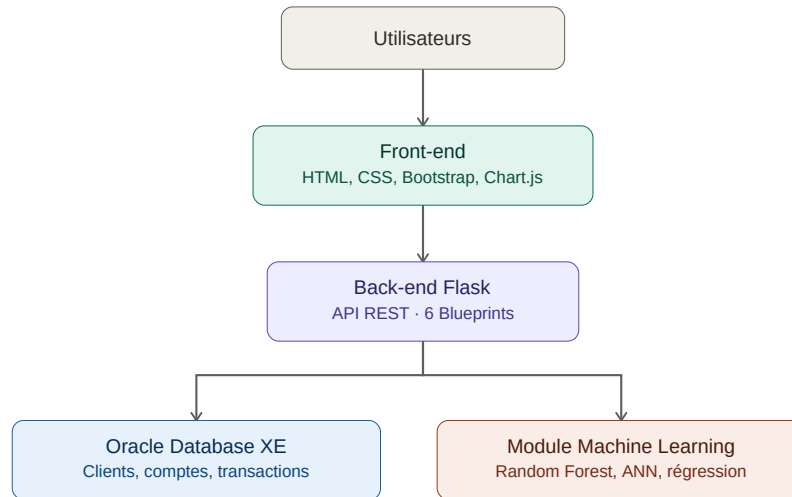


FIGURE 2.1 – Architecture générale de SG SecureBank

gestion des rôles qui permet de réguler les permissions d'accès selon le profil de chaque utilisateur. De plus, l'interface adhère aux principes d'accessibilité et de design adaptatif.

Finalement, l'architecture choisie est élaborée de façon modulaire et susceptible d'évoluer. L'architecture multilayer facilite l'intégration de nouveaux modules ou l'amélioration des modèles d'apprentissage automatique sans nécessiter de modifications sur l'intégralité de l'application. Cette entité facilite aussi une future transition vers une infrastructure cloud en mesure de gérer un volume de transactions plus conséquent.

2.1.1 Brique Data

La composante Data est le noyau central du système intelligent de détection des fraudes. Sa mission consiste à rassembler, organiser, étudier et conserver les données exploitées par les modèles d'apprentissage automatique.

Le traitement des données s'effectue à travers un processus de type ETL (*Extract – Transform – Load*).

La phase initiale (**Extract**) consiste à recueillir les données à partir des formulaires de l'application, des interfaces de programmation d'application REST et des registres d'activité (*logs*). Dans une seconde phase (**Transform**), les données sont épurées puis mises en préparation pour pouvoir être utilisées. Cette étape comprend principalement l'élimination des répétitions, la gestion des valeurs absentes, l'harmonisation des variables numériques, le codage des variables catégoriques et la validation de la consistance des données.

Finalement (**Load**), les données préparées sont mises en oeuvre par les modèles d'apprentissage automatique. Les résultats de prévision, les scores de risque évalués et les alertes produites sont par la suite consignés dans Oracle Database XE pour consultation par les analystes de fraude. Le pipeline complet de traitement des données est présenté dans la Figure 2.2. La base de données simulée de **SG SecureBank** contient **1 050** clients, **1 487** comptes bancaires, **5 205** transactions et **619** alertes de fraude générées.

Le modèle de *Machine Learning* a été entraîné sur l'ensemble des **5 205** transactions, réparties selon une stratégie de séparation des données de **80 %** pour l'apprentissage et **20 %** pour les tests, soit respectivement **4 164** transactions d'entraînement et **1 041**

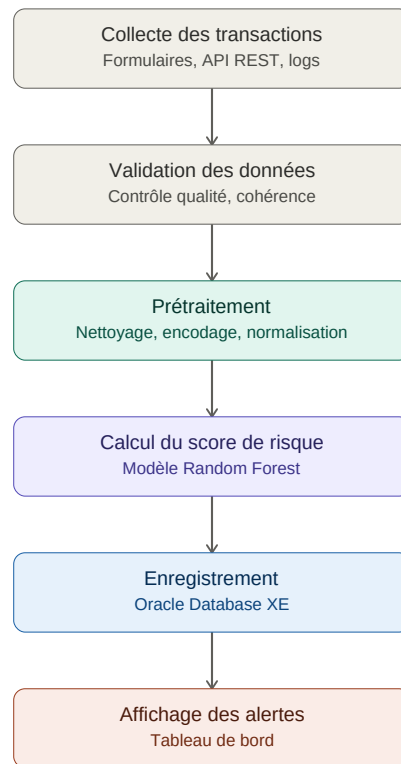


FIGURE 2.2 – Architecture générale de SG SecureBank

transactions de test. Les modèles étudiés dans le cadre du projet sont :

- Régression Logistique .
- Arbre de Décision .
- Random Forest .
- Réseau de Neurones Artificiel (ANN).

Le modèle choisi attribue un indice de risque à chaque transaction pour différencier les opérations usuelles des transactions suspectes. La décision finale est confirmée par un analyste dans le cadre d'une approche *Human-in-the-Loop*.

2.1.2 Collecte et gouvernance des données

Les données exploitées par SG SecureBank proviennent de plusieurs sources complémentaires :

- les formulaires de saisie de l'application .
- les API REST .
- les journaux d'activité (*logs*).

En vue d'une amélioration future, le système pourrait se perfectionner par l'incorporation de sources de données externes, comme les listes actualisées des pays à risque. Le schéma général de collecte et d'exploitation des données est présenté dans la Figure 2.3.

Le modèle d'apprentissage automatique utilise principalement des données relatives au montant de la transaction, à sa catégorie, ainsi qu'aux pays source et destinataire. Le score de risque présent dans les données sert exclusivement à définir l'étiquette de fraude (fraude = score 70) et n'est volontairement pas utilisé comme variable d'entrée du modèle, afin d'éviter toute fuite de données (data leakage) entre la cible et les prédicteurs. Avant d'être utilisées, les données subissent de nombreux tests de qualité pour assurer leur

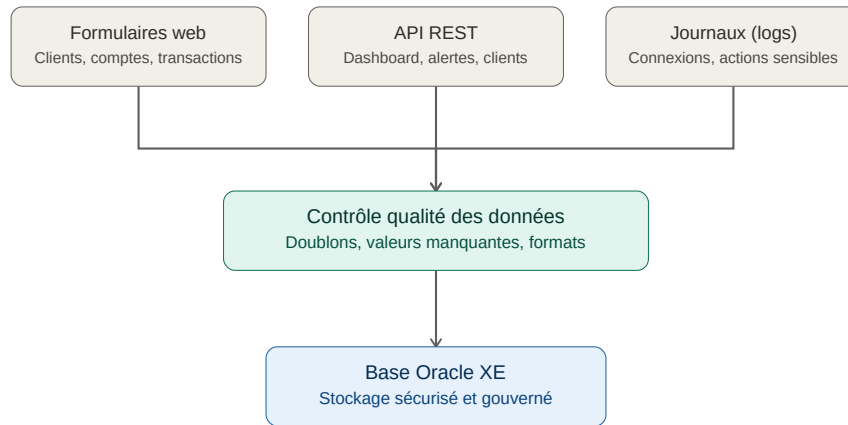


FIGURE 2.3 – Architecture générale de SG SecureBank

fiabilité. Ces vérifications incluent l'élimination des redondances, la gestion des données incomplètes, la confirmation des formats ainsi que la conformité informationnelle. La gouvernance des données repose sur Oracle Database XE, qui assure :

- la gestion des utilisateurs et des rôles .
- le contrôle des droits d'accès .
- la traçabilité des opérations grâce à la journalisation .
- la réalisation de sauvegardes régulières .
- le respect des exigences du RGPD concernant la protection des données personnelles.

Cette organisation garantit la confidentialité, l'intégrité et la disponibilité des données utilisées par l'application.

2.1.3 Justification des choix techniques

Les technologies choisies ont été sélectionnées dans le but de satisfaire les critères fonctionnels, techniques et de sécurité du projet.

Le choix s'est porté sur le framework Flask en raison de sa facilité d'utilisation, de sa modularité et de son adéquation à l'écosystème Python spécialisé dans la Data Science et l'apprentissage automatique. Cela favorise l'élaboration d'API REST efficaces tout en assurant une structuration nette du code.

Oracle Database XE représente une option solide pour le stockage des informations, en raison de ses protocoles de sécurité, sa conformité aux critères ACID, et sa large utilisation dans le domaine bancaire.

Utiliser HTML5, CSS3, Bootstrap 5 et JavaScript pour le développement de l'interface garantit une application réactive, conviviale et adaptée aux principaux navigateurs web.

Les bibliothèques **Chart.js** et **Scikit-learn** offrent respectivement des capacités de visualisation des données et d'intelligence artificielle indispensables pour l'analyse des transactions.

La mise en confrontation de différents modèles d'apprentissage automatique (tels que la Régression Logistique, l'Arbre de Décision, la Forêt Aléatoire et le Réseau de Neurones Artificiels) nous a permis de déterminer la solution qui offre le meilleur équilibre entre précision, vitesse d'exécution et capacité à généraliser.

L'ensemble de ces choix techniques contribue à la création d'une plateforme efficace,

sécurisée, évolutive et correspondant aux exigences de la détection intelligente de la fraude financière.

2.2 Maquettes et prototypes

En ce qui concerne l'élaboration de la plateforme SG SecureBank, une phase de conception d'interface a précédé le développement, avant d'être suivie par la réalisation de captures d'écran de l'application finalisée, présentées ci-après à titre d'illustration de la structure des interfaces, de l'arrangement des informations et des interactions primordiales entre les usagers et le système. À la différence d'un site web traditionnel axé sur le commerce, SG SecureBank est une application professionnelle dédiée aux analystes de fraude. Les interfaces ont ainsi été conçues en se concentrant sur les caractéristiques principales de contrôle, d'évaluation des transactions et de gestion des alertes. Les principales interfaces réalisées sont :

- un formulaire de création d'une nouvelle transaction .
- un tableau d'historique des transactions .
- les informations principales d'une transaction (montant, type, pays, date) .
- le score de risque calculé par le modèle Machine Learning.

2.2.1 Interfaces principales de l'application

Les captures d'écran suivantes illustrent la structure des éléments, la navigation et la mise en page des principales interfaces de la plateforme.

Avant la réalisation des maquettes haute-fidélité, une phase de wireframing a permis de définir la structure générale des interfaces : hiérarchie de l'information, zones fonctionnelles principales et navigation entre les écrans. La figure 2.4 présente les wireframes des principaux écrans de l'application.



FIGURE 2.4 – Wireframes basse-fidélité des écrans principaux : connexion, tableau de bord et transactions

2.2.1.1 Page de connexion

La page de connexion constitue le point d'entrée de la plateforme. Elle permet aux utilisateurs autorisés d'accéder au système après authentification.

Elle contient principalement :

- le logo et l'identité visuelle SG SecureBank .
- les champs de saisie des identifiants .

- le bouton de connexion .
- les messages d'erreur en cas d'échec d'authentification.

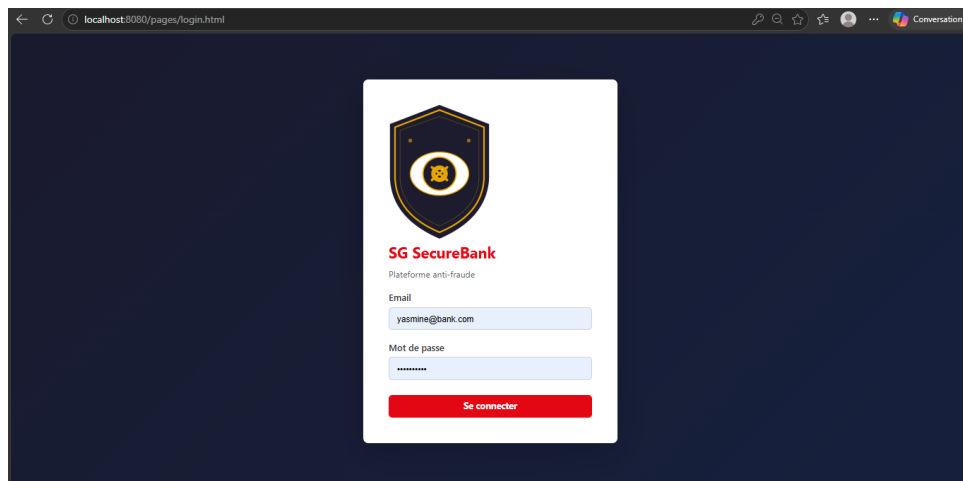


FIGURE 2.5 – Page de connexion

2.2.1.2 Tableau de bord

Le tableau de bord représente l'interface principale utilisée par l'analyste fraude. Il fournit une vue synthétique des activités de surveillance.

Les informations affichées sont :

- nombre total de transactions analysées .
- nombre d'alertes générées .
- fraudes confirmées .
- statistiques des niveaux de risque.

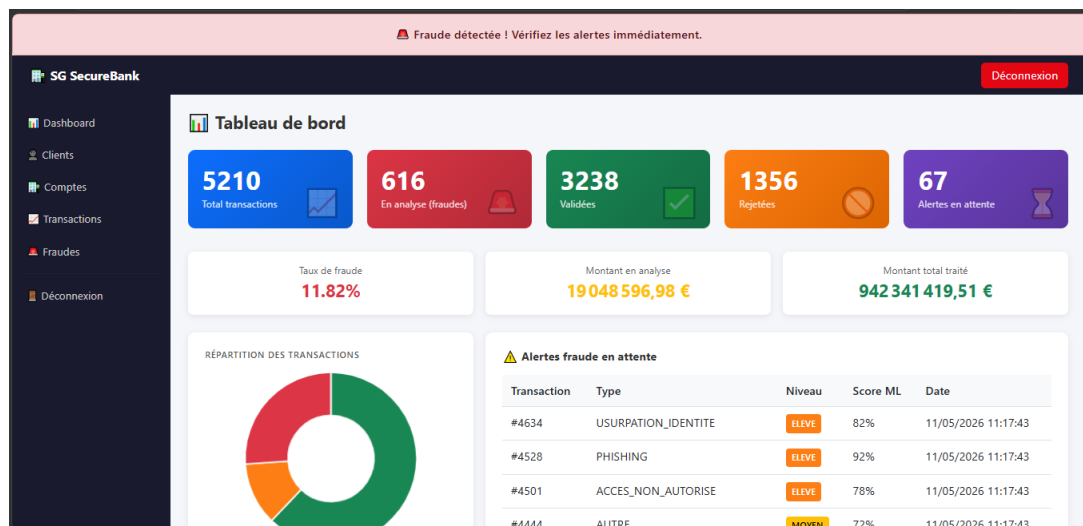


FIGURE 2.6 – Tableau de bord SG SecureBank

2.2.1.3 Page transactions

Cette interface permet aux analystes de consulter l'historique des opérations bancaires et d'identifier rapidement les transactions suspectes.

Elle intègre :

- un formulaire de création d’une nouvelle transaction .
- un tableau d’historique des transactions .
- les informations principales d’une transaction (montant, type, pays, date) .
- le score de risque calculé par le modèle Machine Learning.

#	Type	Montant	Pays	Date	Score risque	Statut
#5289	DEPOT	900 000 009 USD	France → USA	11/06/2026 16:36:53	93%	REJETEE
#5288	RETRAIT	90 000 EUR	France → Tunisie	11/06/2026 16:27:36	100%	REJETEE
#5286	VIREMENT	100 EUR	France → France	11/06/2026 16:27:06	3%	VALIDÉE
#5222	PAIEMENT_EN_LIGNE	5 000 000 EUR	France → Spain	22/05/2026 08:34:52	100%	EN_ANALYSE
#5234	PAIEMENT_EN_LIGNE	5 000 000 EUR	France → Spain	22/05/2026 08:34:52	100%	EN_ANALYSE

FIGURE 2.7 – Page de gestion des transactions

2.2.1.4 Choix graphiques et ergonomiques

Les choix graphiques de l’interface ont été définis selon plusieurs critères UX/UI :

- simplicité d’utilisation .
- hiérarchisation claire de l’information .
- cohérence graphique entre les différentes pages .
- adaptation aux différents formats d’écran.

L’interface utilise une organisation composée de :

- une barre de navigation permettant l’accès aux modules principaux .
- un menu latéral facilitant la navigation .
- une zone centrale dédiée aux données et indicateurs.

La charte graphique repose sur des couleurs adaptées au domaine bancaire :

- le bleu représentant la confiance et la sécurité .
- le blanc améliorant la lisibilité .
- le rouge et l’orange permettant d’identifier les alertes critiques.

2.2.2 Respect des critères ergonomiques

Les maquettes réalisées respectent plusieurs critères ergonomiques essentiels. Le tableau 2.1 présente les principaux critères ergonomiques retenus ainsi que leur application au sein de la solution SG SecureBank.

2.2.2.1 Auto-évaluation et ajustements

Comme le projet se déroulait en solo sans client véritable, l’appréciation de l’ergonomie des interfaces s’est basée sur une auto-évaluation constante, enrichie par les principes d’ergonomie couramment utilisés (visibilité du système, contrôle par l’utilisateur, cohérence graphique, prévention des erreurs et accessibilité).

TABLE 2.1 – Application des critères ergonomiques dans SG SecureBank

Critère ergonomique	Application dans la solution
Visibilité du système	Affichage des indicateurs, scores de risque et statuts des alertes en temps réel.
Contrôle utilisateur	L'analyste conserve la possibilité de valider ou de modifier les décisions proposées par le modèle de Machine Learning.
Cohérence graphique	Utilisation uniforme des composants Bootstrap, des boutons et des couleurs.
Prévention des erreurs	Validation des formulaires et affichage de messages explicatifs lors des actions sensibles.
Accessibilité	Respect des contrastes, lisibilité des textes et navigation claire.

Cette démarche a permis d'identifier plusieurs axes d'amélioration au fil du développement :

- ajout d'un tableau de bord synthétique pour faciliter la prise de décision .
- amélioration de la visibilité des niveaux de risque grâce à un code couleur renforcé .
- repositionnement du bouton « Bloquer compte » directement dans le tableau des alertes afin de réduire le nombre de clics lors du traitement d'une alerte critique .
- simplification de la navigation grâce au menu latéral.

Ces modifications ont donné lieu à une interface plus conviviale, conçue pour satisfaire les besoins des analystes de fraude tout en étant en accord avec les objectifs de fonctionnement de SG SecureBank.

2.2.3 Présentation du développement front-end

2.2.3.1 Choix techniques et justification

Le front-end de SG SecureBank est construit en utilisant HTML5, CSS3 et JavaScript natif, combinés avec le cadre Bootstrap 5.3 et la bibliothèque de visualisation Chart.js. Lors de la phase de surveillance technologique, ces options ont été mises en regard avec d'autres alternatives.

En ce qui concerne les frameworks CSS, une analyse comparative a été réalisée sur Bootstrap, Tailwind CSS et Material UI. Trois raisons principales ont conduit à la sélection de Bootstrap : ses éléments existants (formulaires, tableaux, alertes, badges) facilitant un développement solitaire rapide ; sa capacité à fonctionner directement avec HTML/CSS/JavaScript sans besoin de chaîne de compilation (contrairement à Tailwind qui exige un *build*) ; et sa réputation d'avoir la capacité à créer des interfaces *responsive* appropriées à un environnement professionnel.

La décision d'utiliser JavaScript pur plutôt qu'un cadre front-end (React, Vue) repose sur l'ampleur du projet et le contexte individuel : l'application comporte 6 pages opérationnelles avec des interactions spécifiques (formulaires, tableaux dynamiques, requêtes API REST), ce qui ne requiert pas la complexité supplémentaire d'un framework à base de composants. Chart.js a été intégré pour la représentation graphique des indicateurs du tableau de bord (répartition des transactions selon le niveau de risque).

2.2.3.2 Approche mobile-first

L'interface a été conçue en suivant une démarche *mobile-first* : les styles CSS initiaux sont axés sur l'affichage mobile (navigation horizontale défilante, grille d'indicateurs en une colonne, typographie resserrée). Ensuite, des règles additionnelles sont ajoutées via une *media query min-width* pour améliorer l'affichage à partir des résolutions tablette et desktop (768 px et plus) : la barre de navigation se transforme en un menu fixe latéral, et la grille d'indicateurs se distribue automatiquement sur plusieurs colonnes.

La rupture (*breakpoint*) choisie est 768 px, en accord avec la distinction classique entre mobile et tablette-ordinateur. La disposition des indicateurs dans le tableau de bord se fait grâce à CSS Grid en utilisant `repeat(auto-fill, minmax(200px, 1fr))`. Cela autorise une modification adaptative du nombre de colonnes en fonction de la largeur de l'écran, sans nécessiter de requête média additionnelle.

2.2.3.3 Tests réalisés sur plusieurs résolutions

L'adaptabilité de l'interface a été évaluée sur trois résolutions représentatives : mobile (375 px, similaire à l'iPhone SE/12 mini), tablette (768 px, similaire à l'iPad en mode portrait) et ordinateur de bureau (1440 px). Les figures 2.8, 2.9 et 2.10 illustrent l'apparence du tableau de bord pour chacune de ces résolutions et mettent en évidence l'adaptation de la disposition des composants en fonction de la taille de l'écran.

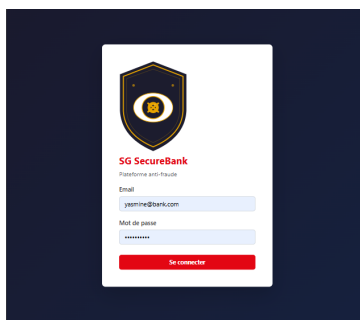


FIGURE 2.8 – Tableau de bord à 375 px (mobile) : navigation en barre horizontale défilante, indicateurs empilés.

Ces vérifications ont été renforcées par l'utilisation des outils de développement du navigateur (mode *responsive*) sur diverses pages de l'application (connexion, clients, comptes, transactions, fraudes), confirmant l'absence de débordement horizontal et l'intelligibilité des tableaux de données à différentes largeurs testées.

2.2.3.4 Limites identifiées

Deux limites ont été identifiées et corrigées au cours du développement :

- l'absence initiale de la balise `<meta name="viewport">` empêchait les navigateurs mobiles d'appliquer les règles *responsive* (la page s'affichait alors comme une version desktop réduite) ; cette balise a été ajoutée sur l'ensemble des pages .
- les tableaux de données (transactions, clients) restent denses sur mobile et nécessitent un défilement horizontal ponctuel ; une évolution future pourrait convertir ces tableaux en cartes empilées liste responsive sous 480 px.

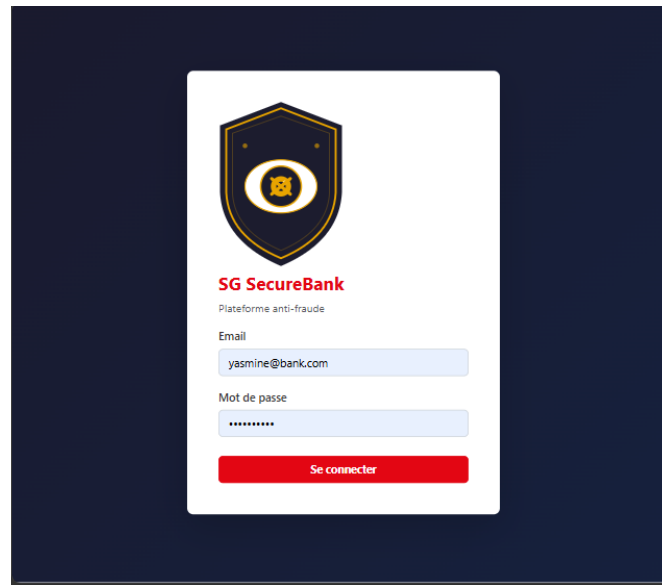


FIGURE 2.9 – Tableau de bord à 768 px (tablette) : bascule vers le menu latéral, grille à 2–3 colonnes.

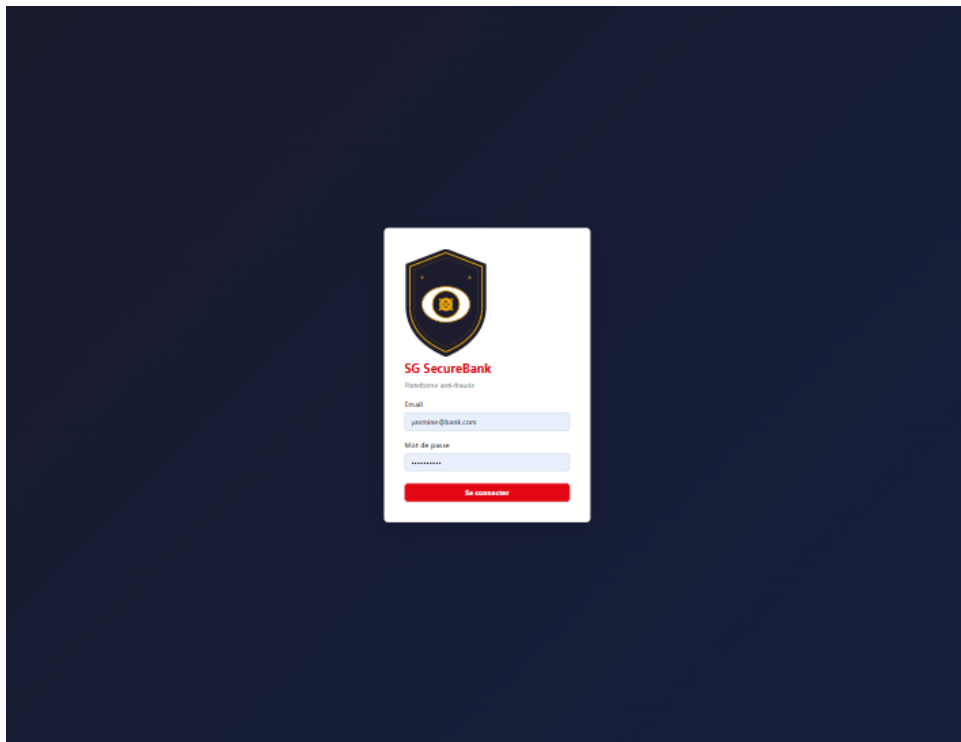


FIGURE 2.10 – Tableau de bord à 1440 px (desktop) : menu latéral fixe, grille d'indicateurs sur 5 colonnes.

2.2.4 La présentation du développement back-end

2.2.4.1 Standards de programmation et bonnes pratiques

L'architecture du back-end est modulaire et suit le modèle Blueprint de Flask. Chaque domaine fonctionnel, tel que l'authentification, les clients, les comptes, les transactions, la fraude et le tableau de bord, est isolé dans son propre fichier de routes. Il y a une distinction nette entre la couche de routes (validation des entrées, code HTTP), la couche de modèle (models.py, définitions SQLAlchemy) et la couche de service (fraud_detector.py,

logique de scoring).

On a systématiquement mis en application les pratiques suivantes :

- validation des champs obligatoires en entrée avec retour 400 explicite en cas de donnée manquante ou mal formée .
- gestion des erreurs par blocs `try/except` avec journalisation (`current_app.logger.exception`) et réponse 500 générique ne divulguant pas la trace d'exécution au client .
- secrets (clé JWT, identifiants Oracle, identifiants admin) chargés exclusivement depuis les variables d'environnement (`.env`), avec échec explicite au démarrage si une variable requise est absente (`_require_env`) plutôt qu'un repli silencieux sur une valeur par défaut faible .
- nommage cohérent des routes et des fonctions en français métier (`valider_fraud`, `bloquer_compte_fraud`), aligné avec le vocabulaire du cahier des charges.

2.2.4.2 Schéma relationnel

Le modèle de données est construit autour de quatre tables liées entre elles par des clés externes, illustrant l'intégralité du parcours d'une transaction : un client peut avoir un ou plusieurs comptes, un compte peut contenir plusieurs transactions, et une transaction peut déclencher une alerte de fraude (figure 2.11).

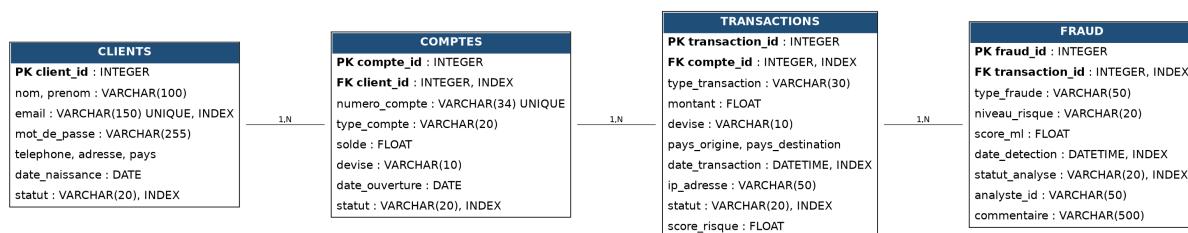


FIGURE 2.11 – Schéma relationnel de SG SecureBank (clés primaires, clés étrangères et index).

Chaque clé étrangère est indexée (`client_id` sur `comptes`, `compte_id` sur `transactions`, `transaction_id` sur `fraud`), de même que les colonnes fréquemment filtrées ou triées (`statut`, `date_transaction`, `date_detection`), anticipant les besoins de performance détaillés en 2.2.4.3.

2.2.4.3 Optimisations des requêtes SQL

Trois optimisations spécifiques ont été mises en uvre :

Consolidation en une unique requête : Le tableau de bord présente sept indicateurs (nombre total de transactions, compteur de fraudes, transactions approuvées/rejetées, montants, alertes en attente). Au lieu de procéder à six requêtes indépendantes de type `COUNT/SUM`, une seule requête SQLAlchemy fusionne les agrégats conditionnels (`SUM(CASE WHEN ...)`) permettant ainsi de réduire les échanges avec Oracle à un seul, au lieu de six.

Pagination systématique : Pour éviter le chargement de l'ensemble d'une table en mémoire lors de chaque requête, les listes (transactions, clients, comptes, fraudes) sont paginées sur le serveur, la limite `per_page` étant fixée à 200 pour prévenir toute utilisation abusive. Le classement basé sur `date_transaction` utilise l'index spécifique plutôt que de se fier à un tri en mémoire au niveau de l'application.

Focus sur les clauses de filtrage dans l'index : Les colonnes `statut` (pour les comptes et les transactions) et `statut_analyse` (pour la fraude) sont indexées en raison de leur usage fréquent dans les clauses `WHERE` des tableaux de bord et des listes filtrées (par exemple : les alertes en cours).

Limitation identifiée : L'appel `GET /api/fraud/pending` récupère la totalité des alertes ayant le statut `EN_COURS` sans pagination (`.all()`), ce qui diffère des autres listes. Ceci est acceptable tant que le nombre d'alertes en attente reste faible, mais devrait avoir une pagination comparable aux autres point d'accès en cas de surcharge de travail.

2.2.4.4 Endpoints sécurisés

La majorité des points d'accès métier (19 sur 20) sont sécurisés par `@jwt_required()` de Flask-JWT-Extended ; seule la route `POST /login` est ouverte au public, ce qui fait sens puisqu'elle représente le point d'entrée de l'authentification. Le jeton JWT devient caduc au bout d'une heure (`JWT_ACCESS_TOKEN_EXPIRES`, modifiable par le biais de `.env`).

De plus, un contrôle d'autorisation est appliqué lors de la réinitialisation du mot de passe : seule l'administrateur ou le client en question peut procéder à cette réinitialisation (par le biais d'une vérification de l'identité JWT par rapport à l'identifiant du client visé). Avant d'être stockés, les mots de passe sont hachés à l'aide de `werkzeug.security.generate_password_hash`. Ils ne sont jamais conservés en format clair dans la base de données.

Le CORS est configuré pour permettre uniquement l'origine du développement frontend (`http://localhost:8080`) et non tous les domaines (*), ce qui limite les requêtes inter-domaines non autorisées.

Limitation détectée : le système de contrôle d'accessibilité ne propose que deux niveaux (administrateur / client authentifié), sans distinguer spécifiquement les rôles analyste fraude et responsable conformité comme spécifié dans les exigences du projet. Un progrès serait d'introduire une colonne `role` dans la table des utilisateurs et de contrôler ce rôle dans les décorateurs de routes.

2.2.4.5 Documentation des API

Le tableau 2.2 recense l'ensemble des points d'entrée exposés par l'API REST.

Chaque réponse suit une convention standard : un objet JSON avec le champ `message` pour les opérations de commande (création, mise à jour, suppression), et un objet de données directement utilisable par le front-end pour les opérations de lecture. Les listes paginées renvoient toujours `items`, `page`, `per_page`, `total` et `total_pages`, ce qui standardise leur utilisation dans `api.js`.

2.2.4.6 Mesures de performance

Un middleware Flask (`before_request/after_request`) mesure le temps de traitement de chaque requête côté serveur. Cette mesure est :

- journalisée dans `performance.log`, au format `méthode chemin -> code_HTTP in temps_ms` ;
- exposée dans l'en-tête de réponse `X-Response-Time-ms`, consultable directement depuis l'onglet réseau des outils de développement du navigateur.

Un script d'analyse (`analyze_performance.py`) compile ces métriques par point de terminaison (nombre d'appels, durée moyenne, minimale, maximale), facilitant l'identification des routes les plus lentes suite à une utilisation concrète de l'application.

TABLE 2.2 – Points d’entrée de l’API REST de SG SecureBank.

Méthode	Route	Auth	Description
POST	/login	Non	Authentification utilisateur et génération du token JWT
POST	/api/reset-password	JWT	Réinitialisation du mot de passe
GET/POST	/api/clients	JWT	Consultation et création des clients
PUT/DELETE	/api/clients/{id}	JWT	Modification et suppression d’un client
GET/POST	/api/comptes	JWT	Gestion des comptes bancaires
PUT/DELETE	/api/comptes/{id}	JWT	Modification et suppression d’un compte
GET/POST	/api/transactions	JWT	Consultation et création des transactions avec détection ML
GET	/api/dashboard	JWT	Récupération des indicateurs du tableau de bord
POST	/api/fraud/detect	JWT	Calcul du score de risque d’une transaction
GET	/api/fraud	JWT	Liste des alertes de fraude
GET	/api/fraud/{id}	JWT	Consultation d’une alerte
PUT	/api/fraud/{id}/valider	JWT	Validation d’une fraude
PUT	/api/fraud/{id}/rejeter	JWT	Classement en faux positif
PUT	/api/fraud/{id}/bloquer	JWT	Blocage du compte associé
PUT	/api/fraud/{id}/reclassifier	JWT	Modification du statut d’une alerte

Observation méthodologique : étant donné que les temps de réponse sont grandement influencés par le matériel, la configuration locale d’Oracle XE et la quantité de données, il est préférable de mesurer les valeurs exactes sur l’environnement de présentation en utilisant l’application pendant quelques minutes, puis en lançant le script mentionné ci-dessus, plutôt que de les présumer à l’avance.

2.2.4.7 Limites identifiées

Trois limites ont été identifiées et documentées pour transparence :

- absence de pagination sur **GET /api/fraud/pending** .
- contrôle d’accès à deux niveaux seulement (administrateur / client), sans les rôles différenciés prévus au cahier des charges .
- les mesures de performance nécessitent une exécution locale de l’application avant la soutenance pour obtenir des valeurs réelles et non estimées.

2.2.5 Tests et qualité logicielle

2.2.5.1 Stratégie de tests

Une suite de tests automatisés a été mise en place avec **pytest**, organisée en trois catégories conformément à la méthodologie retenue : tests unitaires, tests d’intégration

et tests de sécurité. Les tests s'exécutent sur une base de données SQLite en mémoire plutôt que sur l'instance Oracle XE de production. Cette approche permet d'obtenir des exécutions rapides, reproductibles et isolées, sans risque d'altérer les données réelles de l'application.

La suite de tests comprend **38 tests répartis sur cinq fichiers**. Le tableau 2.3 présente l'organisation de cette suite de tests ainsi que les principales fonctionnalités couvertes par chaque fichier.

TABLE 2.3 – Organisation de la suite de tests.

Fichier	Catégorie	Fonctionnalités couvertes
tests/test_auth.py	Intégration et sécurité	Authentification, génération des jetons JWT, comptes suspendus et protection des routes sécurisées.
tests/test_transactions.py	Intégration	Création des transactions, validation des champs obligatoires et pagination.
tests/test_fraud.py	Intégration	Cycle complet de traitement d'une alerte fraude : EN_COURS, CONFIRME, FAUX_POSITIF et RESOLU.
tests/test_fraud_detector.py	Unitaire	Validation de la logique métier du calcul du score (<code>_rule_score</code>) indépendamment de la base de données.
tests/test_security.py	Sécurité	Tentatives d'injection SQL, contrôle d'accès, réinitialisation du mot de passe, non-divulgaration des traces d'erreur et vérification des en-têtes CORS.

Le lancement de la suite de tests s'effectue avec la commande suivante :

```
python -m pytest -q
```

Le résultat obtenu est le suivant :

```
.....
38 passed
```

Tous les tests sont validés avec succès, sans aucun avertissement (0 **warning**). Les avertissements de dépréciation liés à `datetime.utcnow()` et aux anciennes méthodes de SQLAlchemy (`Model.query.get()`) ont été corrigés en remplaçant ces appels respectivement par `datetime.now(timezone.utc)` et `db.session.get()`, conformément aux recommandations de SQLAlchemy 2.x. Le modèle de Machine Learning a également été ré-entraîné avec la version de scikit-learn utilisée en environnement de test, éliminant les avertissements de compatibilité (`InconsistentVersionWarning`) liés à un écart de version entre l'entraînement et l'exécution.

2.2.5.2 Bug détecté et corrigé grâce aux tests

Les tests d'intégration réalisés sur l'API d'authentification (`/login`) ont permis de mettre en évidence une incohérence entre le mécanisme de création des mots de passe et leur vérification.

Lors de la création ou de la réinitialisation d'un mot de passe, l'application utilisait la fonction `generate_password_hash()` de Werkzeug, qui produit un hachage au format

`script`. En revanche, lors de l'authentification, la vérification supposait un format différent (`bcrypt`), ce qui empêchait certains utilisateurs de se connecter et pouvait provoquer une exception lors du contrôle du mot de passe.

La correction apportée dans `app/routes/auth.py` consiste à utiliser exclusivement la fonction `check_password_hash()`, compatible avec les hachages réellement générés par l'application. Une gestion explicite de l'exception `ValueError` a également été ajoutée afin de renforcer la robustesse du traitement.

Cette correction est validée automatiquement par les tests :

- `test_login_success_returns_token`
- `test_self_reset_password_is_allowed`

Limite : Les tests sont exécutés sur une base SQLite en mémoire. Une validation complémentaire sur l'environnement Oracle XE reste recommandée avant un déploiement.

2.2.5.3 Correction d'un second bug

L'analyse du mécanisme `loadComponent()` utilisé pour charger dynamiquement la barre de navigation et le menu latéral a montré que les balises `<script>` insérées via `innerHTML` ne sont pas exécutées automatiquement par le navigateur.

Cette limitation empêchait notamment :

- l'affichage du nom de l'utilisateur connecté dans la barre de navigation ;
- la mise en évidence automatique de la page active dans le menu latéral.

La fonction `loadComponent()` a été modifiée afin de recréer dynamiquement chaque balise `<script>` après son insertion dans le DOM, garantissant ainsi leur exécution.

2.2.5.4 Limites identifiées

Conformément à la priorisation définie selon la méthode MoSCoW, certains éléments n'ont pas été réalisés dans le temps imparti :

- Modification des informations du profil utilisateur via l'interface graphique. La route API est disponible côté serveur mais aucun formulaire n'a été développé côté client.
- Affichage des messages d'erreur directement dans les formulaires. Les erreurs sont correctement renvoyées par l'API mais ne sont pas encore présentées sous les champs concernés.
- Intégration d'un plugin d'accessibilité tiers. Les principales améliorations d'accessibilité ont été réalisées directement dans le code HTML sans recourir à une bibliothèque externe.

Ces éléments constituent des pistes d'amélioration pour une évolution future du projet.

2.2.5.5 Limites des tests

Certaines catégories de tests n'ont pas été mises en uvre dans cette première version du projet :

- absence de tests de charge automatisés .
- absence d'analyse de sécurité automatisée de type SAST ou DAST .
- couverture limitée principalement aux modules `auth`, `transactions` et `fraud`. Les modules `clients` et `comptes`, présentant une logique métier plus simple, n'ont pas fait l'objet de tests spécifiques.

2.2.6 Conformité au RGPD

TABLE 2.4 – Conformité RGPD de l’application.

Exigence	État
Bandeau de cookies	Implémenté : cookies techniques uniquement, choix utilisateur mémorisé dans le navigateur.
Mentions légales	Page dédiée disponible.
Conditions générales d’utilisation	Page dédiée disponible.
Conditions générales de vente	Non applicable.
Gestion des comptes	Suppression disponible ; modification non implémentée côté interface.
Modération des commentaires	Non applicable.

2.2.7 Accessibilité

TABLE 2.5 – Évaluation de l’accessibilité.

Bonne pratique	État
Attribut lang	Présent sur l’ensemble des pages.
Textes alternatifs	Ajoutés aux images informatives.
Contraste	Conforme au niveau WCAG AA après correction.
Labels des formulaires	Ajoutés sur tous les formulaires.
Attributs ARIA	Ajoutés aux principaux éléments interactifs.
Captcha	Non implémenté.
Plugin d’accessibilité	Non implémenté.
Validation W3C	Validation réalisée avec succès (0 erreur).
HTTPS	Prévu pour le déploiement en production.
Responsive Web Design	Implémenté selon une approche mobile-first.

2.2.8 Livrables

Les livrables remis comprennent :

- **Code source back-end** (Flask, Python) : routes API, logique métier, service de détection de fraude, suite de 38 tests automatisés et fichier `requirements.txt`.
- **Code source front-end** (HTML/CSS/JavaScript, Bootstrap) : interface complète intégrant les améliorations d’accessibilité, les pages légales et les correctifs fonctionnels.

L’exécution des tests s’effectue selon les commandes suivantes :

```
cd Backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pytest tests/ -v
```

Les tests utilisent une base SQLite en mémoire et ne nécessitent donc aucune connexion à Oracle XE.

2.2.9 Processus de gestion des mises à jour et des incidents post-déploiement

Une fois la plateforme SG SecureBank déployée, il est nécessaire d'assurer son évolution dans le temps ainsi que la prise en charge rapide des incidents susceptibles de survenir en production. Cette section décrit le plan de mise à jour retenu, le processus de déploiement associé et les scénarios d'alertes définis pour la gestion des incidents.

2.2.9.1 Plan de mise à jour

Les évolutions de la plateforme sont classées en deux catégories, conformément aux pratiques courantes de gestion de versions :

- **Correctifs (patches)** : corrections de bugs, failles de sécurité mineures ou dysfonctionnements identifiés en production (ex. : dépendance obsolète, comportement inattendu d'une route API). Ils sont appliqués dès leur validation par la suite de tests, sans attendre un cycle de publication planifié.
- **Évolutions fonctionnelles** : ajout de nouvelles fonctionnalités ou amélioration de fonctionnalités existantes (ex. : rôles utilisateurs différenciés, voir limite identifiée en 2.2.4.4). Elles suivent un cycle de développement classique (conception, développement, tests, revue) avant intégration.

Le suivi des versions s'appuie sur **Git**, avec une branche principale stable (**main**) protégée, et des branches de travail dédiées à chaque correctif ou évolution. Chaque changement fait l'objet d'un message de commit explicite et, pour les évolutions notables, d'un tag de version (ex. **v1.1.0**) permettant de retracer l'historique des livraisons et de revenir à une version antérieure en cas de besoin.

2.2.9.2 Processus de déploiement

Le déploiement d'une mise à jour suit les étapes suivantes :

1. **Développement** de la correction ou de l'évolution sur une branche dédiée .
2. **Exécution de la suite de tests automatisés** (**pytest**, 38 tests) afin de garantir la non-régression des fonctionnalités existantes .
3. **Revue du code** et validation manuelle des changements .
4. **Déploiement** sur l'environnement de production, en dehors des heures de forte utilisation lorsque cela est possible .
5. **Vérification post-déploiement** : contrôle du bon fonctionnement des principales routes de l'API et du tableau de bord.

En cas d'échec détecté après déploiement, un retour à la version précédente (*rollback*) est effectué grâce à l'historique Git et, si nécessaire, à la restauration d'une sauvegarde de la base de données selon le plan décrit en 1.6.3.

Dans le cadre académique de ce projet, ce processus n'a pas été exécuté sur un environnement de production réel ; il constitue néanmoins la procédure recommandée en vue d'une exploitation effective de la plateforme.

2.2.9.3 Gestion des incidents et scénarios d’alertes

Le suivi de la performance de l’application, déjà mis en place via le middleware de mesure des temps de réponse et le fichier `performance.log`, constitue la base du dispositif de détection d’incidents. Le tableau 2.6 présente les principaux scénarios d’alertes identifiés ainsi que les actions associées.

TABLE 2.6 – Scénarios d’alertes et gestion des incidents.

Scénario	Déclencheur	Action associée
Dégradation du temps de réponse de l’API	Temps de traitement anormalement élevé observé dans <code>performance.log</code> ou l’en-tête <code>X-Response-Time-ms</code> pour une route donnée	Analyse via <code>analyze_performance.py</code> , identification de la route concernée, vérification des requêtes SQL associées
Échec de connexion à la base Oracle XE	Exception levée lors de l’initialisation de la session base de données	Journalisation de l’erreur (<code>current_app.logger.exception</code>), vérification de la disponibilité du service Oracle, redémarrage si nécessaire
Taux d’erreurs 500 élevé	Multiplication des réponses HTTP 500 sur une courte période	Consultation des journaux applicatifs, isolement de la route en cause, application d’un correctif d’urgence si la cause est identifiée
Tentative d’accès non autorisé	Répétition d’échecs d’authentification ou d’accès à une route protégée sans jeton JWT valide	Vérification des journaux d’authentification, blocage temporaire si nécessaire (mesure manuelle dans le cadre académique)
Score de risque anormal du modèle Machine Learning	Proportion de transactions classées à risque significativement supérieure à la distribution habituelle observée dans les données	Vérification de la cohérence des données en entrée, réévaluation du modèle, alerte au responsable technique et conformité

Ces scénarios s’appuient sur les mécanismes de journalisation et de mesure de performance déjà présents dans l’application. Une évolution future pourrait consister à automatiser la détection de ces situations (seuils configurés et notifications automatiques), plutôt que de reposer sur une consultation manuelle des journaux, comme évoqué en 1.3.1.2 concernant les notifications automatiques exclues du périmètre initial.

2.2.10 Retours des parties prenantes et ajustements

Le projet SG SecureBank ayant été mené en solo dans un cadre académique, sans client réel, les retours recueillis proviennent principalement de l’encadrant pédagogique, ainsi que d’une démarche d’auto-évaluation structurée autour des profils utilisateurs simulés (analyste fraude et responsable conformité) définis en 1.2.1. Cette section présente les principaux retours obtenus et les ajustements apportés en conséquence.

2.2.10.1 Retours sur l'organisation et la planification

Le choix initial d'une méthode Scrum classique, envisagée en début de projet, a été discuté avec l'encadrant pédagogique. Ce dernier a souligné qu'une répartition de rôles (Scrum Master, Product Owner, équipe de développement) n'avait pas de sens pour un projet porté par une seule personne. À la suite de ce retour, l'organisation a été ajustée vers une approche Kanban personnelle, plus adaptée au suivi individuel des tâches via Jira, tout en conservant les principes agiles de priorisation continue et d'adaptation aux imprévus.

2.2.10.2 Retours sur l'interface et l'ergonomie

En l'absence d'utilisateurs réels pour tester la plateforme, l'évaluation ergonomique s'est appuyée sur une auto-évaluation continue au regard des critères présentés en 2.2.2 (visibilité du système, contrôle utilisateur, cohérence graphique, prévention des erreurs, accessibilité). Cette démarche a mis en évidence plusieurs axes d'amélioration, qui ont conduit aux ajustements suivants :

- ajout d'un tableau de bord synthétique, jugé initialement insuffisant pour une prise de décision rapide par l'analyste fraude .
- repositionnement du bouton « Bloquer compte » directement dans le tableau des alertes, afin de réduire le nombre de clics nécessaires lors du traitement d'une alerte critique .
- renforcement du code couleur pour améliorer la visibilité des niveaux de risque .
- simplification de la navigation grâce à l'ajout d'un menu latéral.

2.2.10.3 Retours sur la sécurité et la conformité

L'analyse critique du code, réalisée en cours de développement, a permis d'identifier plusieurs points de vigilance en matière de sécurité, traités comme des retours internes avant leur correction :

- des identifiants administrateur codés en dur dans `auth.py` ont été déplacés vers les variables d'environnement (`.env`) ;
- la route `/api/reset-password`, initialement accessible sans authentification, a été protégée par `@jwt_required()` ;
- la configuration CORS, initialement ouverte à toute origine, a été restreinte à l'origine du front-end de développement .

Ces ajustements ont également conduit à l'ajout du fichier `tests/test_security.py`, afin de garantir que ces corrections restent valides lors des évolutions futures du code.

2.2.10.4 Retours sur la conformité RGPD

Une relecture du cahier des charges au regard des exigences RGPD a mis en évidence l'absence initiale de dispositifs pourtant attendus pour une application manipulant des données bancaires. Cela a conduit à l'ajout, en fin de développement, du bandeau de cookies, des pages de mentions légales et des conditions générales d'utilisation .

2.2.10.5 Synthèse

L'ensemble de ces retours, bien qu'issus d'un contexte académique sans partie prenante externe au sens strict, a permis d'orienter le projet vers une solution plus conforme aux attentes exprimées dans le cahier des charges, en particulier sur les volets ergonomie, sécurité et conformité réglementaire. Certains ajustements identifiés n'ont cependant pas

pu être intégralement traités dans le temps imparti, comme la distinction des rôles « analyste fraude » et « responsable conformité » au niveau du contrôle d'accès , et constituent des pistes d'évolution pour la suite du projet.

Annexes

A.1 Personas

A.1.1 Persona 1 : Analyste fraude

Nom fictif : Camille Durand

Rôle : Analyste anti-fraude au sein du service conformité

Objectifs : Identifier rapidement les transactions à risque, réduire le temps de traitement des alertes, éviter les faux positifs

Frustrations : Manque de centralisation des informations, absence d'indicateurs visuels clairs pour prioriser les alertes urgentes

Besoins : Tableau de bord synthétique, historique des transactions consultable rapidement, workflow de validation simple.

A.1.2 Persona 2 : Responsable conformité et sécurité

Nom fictif : Karim Bensaïd

Rôle : Responsable technique et conformité RGPD

Objectifs : Garantir la conformité réglementaire, superviser la sécurité des accès et des données.

Frustrations : Difficulté à obtenir une traçabilité complète des actions sur les alertes.

Besoins : Journalisation des actions, gestion fine des rôles et permissions, rapports de suivi.

A.2 Compte-rendu d'atelier simulé

Contexte : en l'absence de client réel, un atelier de co-création a été reconstitué à partir des exigences du cahier des charges et des pratiques courantes du secteur bancaire, afin de simuler une démarche professionnelle de recueil des besoins.

Participants (simulés) : porteur de projet (rôle chef de projet), encadrant pédagogique (rôle client/sponsor).

Objectif de l'atelier : identifier les fonctionnalités prioritaires du futur système de détection de fraude et valider le périmètre fonctionnel.

Conclusions principales :

- Le système doit permettre une détection automatique tout en conservant une validation humaine (approche Human-in-the-Loop).
- Le tableau de bord doit fournir une vue synthétique immédiatement exploitable par l'analyste.

- La traçabilité des décisions est jugée indispensable pour la conformité réglementaire.

A.3 Cas d'utilisation principaux

UC1 – Authentification

Acteur : tout utilisateur. L'utilisateur saisit ses identifiants ; le système vérifie les informations et génère un jeton JWT en cas de succès.

UC2 – Création d'une transaction

Acteur : analyste fraude. L'utilisateur saisit les informations d'une transaction (montant, type, pays) ; le système calcule automatiquement un score de risque via le modèle Random Forest.

UC3 – Traitement d'une alerte de fraude

Acteur : analyste fraude. L'utilisateur consulte une alerte générée, analyse les informations de la transaction associée, puis valide la fraude, la classe en faux positif, ou la marque comme résolue.

UC4 – Blocage d'un compte

Acteur : analyste fraude. Suite à la confirmation d'une fraude, l'utilisateur déclenche le blocage du compte concerné.

UC5 – Consultation du tableau de bord

Acteur : analyste fraude, responsable conformité. L'utilisateur consulte les indicateurs globaux (transactions analysées, alertes, taux de fraude).

Conclusion générale

Le projet SG SecureBank avait pour objectif de concevoir et développer une application web intelligente destinée à assister les analystes fraude dans la détection et le traitement des transactions bancaires suspectes, en combinant des modèles d'apprentissage automatique à une validation humaine selon une approche Human-in-the-Loop.

La démarche suivie a couvert l'ensemble du cycle de développement d'un projet digital : analyse des besoins et rédaction d'un cahier des charges, veille technologique et choix des solutions techniques, conception de l'architecture en trois couches, développement du front-end et du back-end, intégration d'un module de Machine Learning, mise en place d'une suite de tests automatisés, ainsi que la prise en compte des exigences de sécurité, de conformité RGPD et d'accessibilité.

Sur le plan technique, la solution retenue s'appuie sur un back-end Flask structuré en Blueprints, une base de données relationnelle Oracle XE organisée autour de quatre tables liées par des clés étrangères, et un modèle Random Forest chargé de calculer un score de risque pour chaque transaction. La suite de 38 tests automatisés, couvrant l'authentification, les transactions, la gestion des alertes et la sécurité, a permis de détecter et de corriger deux anomalies significatives au cours du développement, illustrant l'apport concret d'une démarche de tests rigoureuse.

Le projet a également mis en évidence plusieurs limites, documentées de façon transparente tout au long du rapport : un contrôle d'accès à deux niveaux ne distinguant pas encore les rôles « analyste fraude » et « responsable conformité » prévus au cahier des charges, l'absence de pagination sur un point d'accès spécifique, ainsi qu'une couverture de tests non exhaustive sur l'ensemble des modules. Ces limites, loin de remettre en cause la solution développée, constituent des pistes claires d'amélioration pour une évolution future de la plateforme.

Réalisé dans un cadre académique, sans client ni utilisateur final réel, ce projet a nécessité de reconstituer une démarche professionnelle complète : recueil de besoins simulé, auto-évaluation ergonomique continue, et prise en compte de retours obtenus principalement auprès de l'encadrement pédagogique. Cette contrainte a également permis de développer une capacité d'autonomie et de recul critique sur les choix techniques effectués, indispensable dans un contexte professionnel réel.

À terme, une mise en production effective de SG SecureBank dans un environnement bancaire nécessiterait un renforcement des exigences de sécurité et de disponibilité, une infrastructure d'hébergement dédiée, ainsi que l'intégration des évolutions identifiées tout au long de ce rapport, notamment la différenciation des rôles utilisateurs et l'automatisation de la détection des incidents. Ce projet constitue néanmoins une base solide, fonctionnelle et testée, démontrant la faisabilité d'un système de détection de fraude alliant intelligence artificielle et supervision humaine.

Bibliographie

[1] Société Générale. Résultats au 31 mars 2026 communiqué de presse, 2026.